

Graph-Based Energy Budget Computation in Wireless Sensor Networks

Bachelorarbeit

by

Roman Riazanov

Matrikel-Nr.: 23152805

Supervision:

Prof. Dr.-Ing. Jürgen Teich
M.Sc. Pierre-Louis Sixdenier

March 24, 2026

Contents

1	Introduction	1
2	Literature Review	5
2.1	Energy Budget Management in WSNs	5
2.2	Graph-based Path Finding Algorithms	19
3	Fundamentals	25
3.1	System Model	25
3.1.1	Energy Neutrality	25
3.1.2	Tasks	26
3.2	Energy Model	27
3.2.1	Energy Harvesting System	27
3.2.2	Battery Model	27
3.2.3	Energy Harvesting Prediction	29
3.3	Problem Description	30
4	Methodology	31
4.1	Graph Model	31
4.2	A* Algorithm	33
4.3	Topological Sorting	33
4.4	Beam Search	34
4.5	Monte-Carlo Tree Search based Algorithm	37
4.6	Recalculation	38
5	Results	41
5.1	Simulation System Parameters	41
5.2	Optimal Algorithm Parameters	41
5.3	Performance	46
5.4	Recalculation Results	51
5.5	Operation Simulation	53
6	Conclusion	55
A	Appendix	57
A.1	Beam Search and MCTS Graph Examples	57

Contents

A.2	Operation Simulation Results	60
A.3	Duty Cycle based Tasks	62
A.4	Penalty function parameter k	63
Bibliography		65

Bachelorarbeit

“Graph-Based Energy Budget Computation in Wireless Sensor Networks”

Student: Roman Riazanov
Supervisor: Prof. Dr.-Ing. Jürgen Teich
Contact: M.Sc. Pierre-Louis Sixdenier

Fundamentals

In Wireless Sensor Networks (WSNs), sensor nodes are most often powered by batteries which have limited capacity. Environmental energy can be harvested from sources such as solar, or kinetic energy to extend the operational lifetime of these nodes. However, the harvested energy is typically intermittent and unpredictable, making it challenging to ensure continuous operation of the sensor nodes. The energy consumption of the nodes must be managed in order to match the harvested energy, while maximizing the quality of service (QoS) provided by the WSN. This mode of operation is known as energy-neutral operation (ENO) [6].

For solar energy harvesting WSNs, approaches exist that exploit the pseudo-periodicity of the energy source (i.e., day-night cycles) to compute a daily energy budget that guarantees ENO [3, 7, 8], given a predicted amount of harvested energy throughout the day. However, solving this problem is costly, especially on resource-constrained sensor nodes, and the computed energy budget may not be optimal due to inaccuracies in the estimation of the amount of energy harvested. Furthermore, some ENO constraints (e.g., that the battery level at the end of the day shall be greater or equal than the battery level at the start of the day) are soft constraints, i.e., solutions that slightly violate these constraints may be acceptable if they provide a better QoS or are obtained with lower computational costs.

Thesis Description

In this thesis, graph-based methods to approximately compute energy budgets for solar energy harvesting WSNs shall be investigated. Graphs have been successfully applied to long-term decision-making problem, such as in Markov Decision Processes (MDPs) [1], and in combinatorial game theory using game trees [2].

This problem could be modeled as a single directed acyclic graph (DAG) of depth N , where N is the number of time slots in a day (e.g., 24 for hourly slots). An example of such a DAG is shown in Figure 0.1.

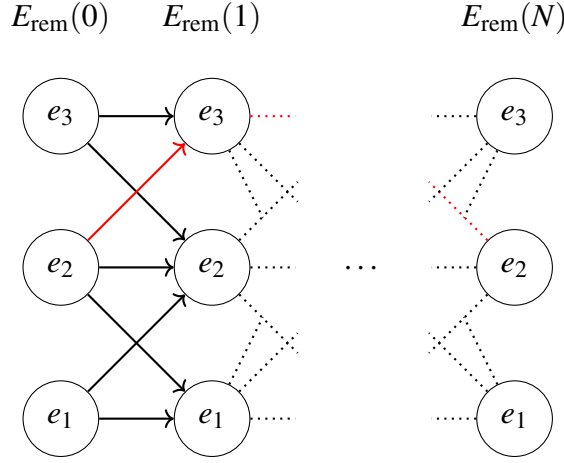


Figure 0.1: An example of a directed acyclic graph that can be used to determine an energy budget for a solar energy harvesting WSN. Each node represents a state-of-charge of the battery at the start of a time slot ($E_{\text{rem}}(n) \in \{e_1, e_2, e_3\}$), while each edge represents a possible energy consumption decision (e.g., consuming an amount of energy b_1 , b_2 , or b_3) that leads to a new state-of-charge at the next time step, taking into account the predicted harvested energy. Highlighted in red is a path from the root node (i.e., initial battery level $E_{\text{rem}}(0)$) to a leaf node (i.e., end of the day $E_{\text{rem}}(N)$), of which the edges' labels represent an energy-neutral budget for the day. Note that different energy consumptions may lead to the same state-of-charge, hence the merging of some nodes.

The cost of a path is defined as a sum of all the weights of the edges in the path (i.e., the amount of energy allocated per time slot) and a penalty for violating ENO constraints. A graph only holds for a certain predicted harvested energy profile and has to be explored every time the predicted harvested energy changes. To find the longest path in a timely manner, graph traversal heuristics, such as Monte Carlo Tree Search (MCTS) [4], can be used.

The following questions shall be addressed in this thesis:

- RQ1 What are the most suitable graph traversal heuristics to compute an approximate solution to the energy budget computation problem?
- RQ2 How do the proposed graph-based approaches compare to existing methods (such as [3, 5]) in terms of computation time, memory footprint, and quality of the computed energy budget?

As part of this thesis, the student will perform the following tasks:

- Acquisition of the necessary background knowledge on energy-neutral operation in solar energy harvesting WSNs, and on graph-based heuristics for long-term decision-making problems.
- Implementation of a model of a graph-based model of the energy budget computation problem. In addition to the exact solver, at least one graph traversal heuristic shall be implemented to compute an approximate solution to the problem.
- Implementation of the algorithms on an nRF52 Development Kit and measurement of its energy consumption using a Nordic Power Profiler Kit II.
- Evaluation of the algorithms in terms of quality of the computed energy budgets and of energy constraints violation, and comparison with existing methods (such as [3, 5]). These evaluations will be conducted while varying hyperparameters of the obtained approach (e.g., granularity of the graph representation) to study the trade-offs between computation time, memory footprint, and quality of the computed energy budget.
- Preparation of the written thesis, documentation of all programs, and archiving of the relevant data in a repository.

Resources Used

An existing Python simulator of a solar energy harvesting sensor node will be provided to the student. The implementation on a resource-constrained microcontroller will be done using an nRF52 Development Kit¹, of which the energy consumption will be measured using a Nordic Power Profiler Kit II².

- [1] Hub Ali, Dawei Gong, Meng Wang, and Xiaolin Dai. “Path Planning of Mobile Robot With Improved Ant Colony Algorithm and MDP to Produce Smooth Trajectory in Grid-Based Environment”. In: *Frontiers in Neurorobotics* Volume 14 - 2020 (2020). ISSN: 1662-5218. DOI: 10.3389/fnbot.2020.00044. URL: <https://www.frontiersin.org/journals/neurorobotics/articles/10.3389/fnbot.2020.00044>.

¹<https://www.nordicsemi.com/Products/Development-hardware/nRF52-DK>

²<https://www.nordicsemi.com/Products/Development-hardware/Power-Profiler-Kit-2>

- [2] Darse Billings, Aaron Davidson, Terence Schauenberg, Neil Burch, Michael H. Bowling, Robert C. Holte, Jonathan Schaeffer, and Duane Szafron. “Game-Tree Search with Adaptation in Stochastic Imperfect-Information Games”. In: *Computers and Games, 4th International Conference, CG 2004, Ramat-Gan, Israel, July 5-7, 2004, Revised Papers*. Ed. by H. Jaap van den Herik, Yngvi Björnsson, and Nathan S. Netanyahu. Vol. 3846. Lecture Notes in Computer Science. Springer, 2004, pp. 21–34. DOI: 10.1007/11674399_2. URL: https://doi.org/10.1007/11674399%5C_2.
- [3] Antonio Caruso, Stefano Chessa, Soledad Escolar, Fernando Rincón, and Juan C. López. “Task Scheduling Stabilization for Solar Energy Harvesting Internet of Things Devices”. In: *2022 IEEE Symposium on Computers and Communications (ISCC)*. ISSN: 2642-7389. June 2022, pp. 1–6. DOI: 10.1109/ISCC55528.2022.9913061. URL: <https://ieeexplore.ieee.org/document/9913061> (visited on 05/28/2024).
- [4] Raphael Chekroun, Thomas Gilles, Marin Toromanoff, Sascha Hornauer, and Fabien Moutarde. “MBAPPE: MCTS-Built-Around Prediction for Planning Explicitly”. In: *2024 IEEE Intelligent Vehicles Symposium (IV)*. 2024, pp. 2062–2069. DOI: 10.1109/IV55156.2024.10588457.
- [5] Lars Hanschke and Christian Renner. “EmRep: Energy management relying on state-of-charge extrema prediction”. en. In: *IET Computers & Digital Tech* 16.4 (July 2022), pp. 91–105. ISSN: 1751-8601, 1751-861X. DOI: 10.1049/cdt2.12033. URL: <https://onlinelibrary.wiley.com/doi/10.1049/cdt2.12033> (visited on 10/29/2024).
- [6] Aman Kansal, Jason Hsu, Sadaf Zahedi, and Mani B. Srivastava. “Power management in energy harvesting sensor networks”. In: *ACM Transactions on Embedded Computing Systems* 6.4 (Sept. 2007), 32–es. ISSN: 1539-9087. DOI: 10.1145/1274858.1274870. URL: <https://dl.acm.org/doi/10.1145/1274858.1274870> (visited on 04/06/2024).
- [7] Pierre-Louis Sixdenier, Stefan Wildermann, Jebacyril Arockiaraj, and Jürgen Teich. “WiP Paper: Utility-Aware Transmission of Sensor Data on Energy-Harvesting IoT Gateways”. In: *22nd International Conference on Embedded Wireless Systems and Networks (EWSN 2025)* (Leuven, Belgium). Sept. 22–25, 2025.
- [8] Pierre-Louis Sixdenier, Stefan Wildermann, and Jürgen Teich. “GRES: Guaranteed Remaining Energy Scheduling of Energy-harvesting Sensors by Quality Adaptation”. In: *13th Mediterranean Conference on Embedded Computing, MECO 2024, Budva, Montenegro, June 11-14, 2024*. IEEE, 2024, pp. 1–5. DOI: 10.1109/MECO62516.2024.10577838. URL: <https://doi.org/10.1109/MECO62516.2024.10577838>.

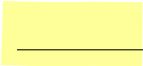
1 Introduction

Wireless Sensor Networks (WSNs) are a big and ever-growing part of the rapidly expanding Internet of Things (IoT), which is a term generally referring to devices that are not usually considered computers, but possess network connectivity and are able to exchange data with other systems [59]. In this context, WSNs act as an important interface between the physical world and digital information systems by offering distributed sensing capabilities. WSNs are usually made up of many individual nodes, each of which has the ability to perform one or more sensing tasks, as well as basic computational operations. These nodes are often deployed in large numbers and distributed across big areas. Typical applications include the measurement of environmental values such as temperature, humidity, air pressure, air quality, or seismic activity in order to support agriculture, observe the climate, or predict natural-disasters. In the future, WSNs are expected to become even more prevalent through their integration into smart homes and smart cities, where they can, for example, monitor key infrastructure like bridges [30, 21]. A single node in a WSN does not only have to take measurements, but also has to process and transmit the recorded data to a sink node or base station, where the information is collected and forwarded for further analysis. In some scenarios, nodes may even perform parts of the evaluation themselves, for example, by filtering, compressing, or aggregating sensor readings before transmission, in order to reduce communication overhead. All of these operations, sensing, computation, and wireless communication, require energy, which is one of the most significant bottlenecks in modern WSN applications.

Nodes in WSNs are usually equipped with a battery to supply them with energy. However, this battery will eventually run out, at which point the node goes offline and stops contributing to the network. In many real-world deployments, replacing or recharging batteries is not economically viable due to the large number of nodes or the effort required. In other cases, battery replacement may be practically impossible because of the deployment location, for example, when sensor nodes are embedded in structures, scattered across remote places, or placed in extreme environments such as inside volcanoes [30, 38]. Under such conditions, the lifetime of a node is effectively limited by how long its battery lasts until depletion. This fundamental problem has been addressed in numerous research works that propose energy-aware algorithms and system designs aimed at keeping nodes operational for as long as possible [65, 35]. Since supplying nodes with a permanent external power source is generally infeasible due to their sheer number and distributed nature, energy harvesting has been proposed as a promising alternative to extend the time a node can

operate. Early examples include platforms such as the Heliomote [36], and the concept has also been explored in a variety of other studies [58, 57]. Multiple different energy sources and harvesting techniques exist, ranging from widely used solar and wind energy harvesting to less common methods such as harvesting mechanical vibrations [21] or electrochemical energy produced by bacteria [64]. Although these sources can provide a continuous or semi-continuous stream of energy, their output is typically highly variable and difficult to predict due to changing environmental conditions, night and daytime in solar energy harvesting being the most prominent example. Consequently, in order for a node to remain operational during periods of low or no incoming energy, the available energy budget must be carefully managed. Efficient energy management is therefore a central challenge in energy-harvesting WSNs and a key requirement to achieve long-term reliable operation.

On the other hand, the quality of work a node performs is also very important. The abstract term "quality" or also "Quality of Service" (QoS) can be defined as sensing frequency, sensing accuracy (if the node has the possibility to use different methods or sensors for measurements) or communication frequency [15], but can also be influenced by different implementations of algorithms or different inputs for the algorithms [60]. A common point that these definitions have, is that the higher the quality of service of the performed work, the more energy it consumes. Thereby a two-faced problem arises. While as little energy as possible should be used, in order for the battery not to run out and the node to go offline, the node should also **work as "good" (QoS-wise) as possible**. This essentially means that a node **cannot just use** the lowest-energy consuming options for every implementation consideration, but some kind of tradeoff needs to be made between energy consumption and QoS. Many papers exist, addressing this exact topic, with them having different models for their systems, their energy harvesting aspects and the work the node performs. In this paper a model for the work similar to [9] is used, where a node has different tasks it can perform, all having different energy cost and associated quality. **They are, furthermore,** completely independent of any functionality, meaning different tasks are independent of each other and there are no mandatory tasks. In order to guarantee that the node stays online as long as possible, the energy-neutrality framework first introduced by Kansal et al. in [31] is used, which states, that if the battery of a node always has at least the same charge at the start of a time span as at the end and a defined minimum is never exceeded (so constant node operation is ensured), the node can in theory operate for an unlimited time. The time span is further divided up into discrete time slots, to which an exact duty cycle is assigned. A duty cycle can be described as a certain level of work. In this paper however, one task is assigned per time slot. Overall, the goal is to find an assignment of tasks to time slots, such that: (1) **the assignment is energy neutral** in regards to [31], (2) **the battery charge never goes below a pre-defined minimum** and (3) **the summed up qualities of the tasks is as high as possible**. In order to find this assignment, energy harvesting predictions for the next time span are assumed to be known at the start of a time span. Since



such predictions, as already stated are rarely completely accurate, a recalculation of the remaining task assignments, possibly with new harvesting predictions can also be performed in the middle of the time span to ensure the goals are met.

As an approach, it was chosen to represent this problem in a graph based model, in contrast to other works, where the same or a similar problem is introduced and described as an Integer Linear Program (ILP) [9] or optimization problem [26]. Using the graph based model, it is then possible to apply existing graph exploration algorithms to find solutions to the energy budget management problem. These algorithms may provide an exact solution, however, the main part of this thesis focuses on heuristics, that offer a good, imperfect solution, but may have advantages in other categories, similar to the algorithm proposed in [15]. To the best of my knowledge, this is the first thesis investigating this problem using a graph based approach.

Specifically, the goals for this thesis is finding answers to the following research questions:

1. What are the most suitable graph traversal heuristics to compute an approximate solution to the energy budget computation problem?
2. How do the proposed graph-based approaches compare to existing methods (such as [9], [15]) in terms of computation time, memory footprint, and quality of the computed energy budget?

To achieve this, this thesis follows the structure outlined here. First, a comprehensive literature review examines WSNs and the problem space of energy management and identifies the state of the art in existing algorithms addressing this challenge. This is followed by a brief evaluation of established graph traversal algorithms and heuristics. Next, the models and design choices adopted in this thesis are presented and the precise problem to be solved is formally defined. Subsequently, the selected algorithms and heuristics are introduced and their adaptation to the problem is described. Finally, their performance is evaluated using a range of metrics and the thesis concludes with a discussion of the results.



2 Literature Review

2.1 Energy Budget Management in WSNs

There are many papers mentioning Wireless Sensor Networks (WSNs) and talking about different topics regarding them. A WSN, according to [3] is a network of wirelessly communicating sensors that are used to measure some environmental variable. In addition to sensors, this network may include base stations or sink nodes to which the sensors send their data, as well as gateways to other networks. Moreover, the authors show a basic architecture for a sensor node, consisting of hardware units for sensing, processing, transceiving and powering the node.

One of the earliest mentions of Wireless Sensor Networks was by Pottie [55], talking about various design considerations, like node communication and already mentioning the critical point of power management in the network. He specifically talks about Wireless Integrated Network Sensors (WINSs), which are sensors directly integrated into an existing network infrastructure. This is extended by the later work [56], where they go into more detail regarding design considerations and especially emphasize the limited energy in nodes and communication being the largest energy consumer. They propose a processing hierarchy, where a node stays in a low power state for as long as possible and only uses higher power state if it detects that it is necessary or the state was manually changed. Moreover, they communicate the idea of doing as much computation locally in a node as possible, as the communication of data to a sink node would use much more energy, especially if a node uses an application-specific integrated circuit (ASIC) in contrast to a general-purpose processor.

To investigate in which use cases WSNs can be applied, Kandris et al. [30] conduct an exploration of WSN applications. They list categories similar to [3], namely military use, healthcare, environmental applications, Flora and Fauna applications, industrial use and urban applications. For each of these categories they provide sub-categories and examples, with a few examples being displayed in Table 2.1. A unique type of WSN, which does not directly fit into any of the listed categories, is the Wireless Multimedia Sensor Network (WMSN). Other than typical WSNs, that have some type of specific sensor and send small data packages to a target node, WMSNs record multimedia content, such as audio, images and even videos, that can be evaluated. Therefore, as shown in [2], they themselves have a wide range of possible use cases and rather form a subgroup of WSNs, than being a category. Some challenges of WMSNs the paper describes are needing a high bandwidth for data transmission,

as multimedia data is usually much larger than data recorded by simple sensors and necessarily needing a constant stream of data, which means the node has to operate constantly. Moreover, processing multimedia data is much harder, which makes large processing hubs a requirement and uses a lot of energy.

Category	Ex. for Subcategory	Ex. for application	Sensing value
Military	Battlefield Surveillance	Ground Surveillance	Magnetic/Acoustic Signals
Health	Home Assisting Systems	Patient Monitoring	Vital Signs
Environmental	Emergency Alerting	Seismic Sensor	Seismic activity
Flora & Fauna	Crop Monitoring	Fertilization System	Moisture/Temperature
Industrial	Logistics	Transportation Condition	Temperature/Humidity
Urban	Structural Monitoring	Bridge Health	Acceleration

Table 2.1: Examples for WSN applications presented in [30].

As has become obvious, the most valuable, but also most limited resource in a sensor node is energy. As a consequence, energy in sensor nodes must be managed efficiently. An early paper talking about energy management in WSNs focuses on nodes which have a battery as their energy supply is [65]. In order to extend the time until the battery runs out as long as possible, systems could simply operate at the lowest energy-consuming level possible. While easy to implement, this approach has some major downsides. On the one hand, for most sensor applications, there are long phases where there is nothing significant or new to measure or sense, therefore any energy used to take measurements during these time spans is wasted. On the other hand, during phases where there are a lot of events or changes to detect, the node would still operate at a low level, which may negatively impact crucial measurements and may even lead to the node missing events completely. Thereby, this constantly low operation level would not only waste energy, but also heavily worsen the quality of work that is conducted by the node. As a solution, the authors introduce Dynamic Power Management, which would increase the nodes operation level if many measurements are necessary and decrease it, if there is nothing to measure. As a technique to control the performance and energy required by the node, they propose Dynamic Voltage Scaling (DVS) for the CPU, which means that the operating voltage and frequency for the CPU is scaled based on the desired performance level. In order to know when and how to scale the performance, they develop a workload prediction strategy based on adaptive filtering of past workload data and further test different filtering methods for this strategy. Finally, they conclude that this can reduce energy consumption in a node by up to 3-4 times.

A similar problem is investigated by Kulau et al. [35]. In order to modify the energy consumption, they choose to adapt the sampling rate. A higher sampling rate is

therefore used when many changes that can be measured are detected and a low rate is used during phases without significant changes in the environment. For means of predicting the required sampling rate for a point in time, they present a problem-specific adaptation of Bollinger Bands, an analysis tool originally developed to analyze stock prices, which is used to filter historic data. In conclusion, their approach results in minimal measurement inaccuracies, while significantly reducing energy consumption.

Looking at this problem from a network perspective, there are even more possibilities to better manage available energy budgets. The papers [62, 74] aim to optimize energy usage and system uptime for entire sensor networks, by adjusting data routing inside the networks. Shah and Rabaey first show, that the until then commonly used approach of using the lowest energy path to route data is not always suitable to ensure network operating time and connectivity, because certain nodes get used more often, which leads to them running out of energy and can thereby cause network partitions. Their idea is to occasionally use suboptimal paths to ensure that the data traffic and consequently the energy consumption is distributed more evenly over the entire network. In [74] however, a centralized approach is proposed, where sensor networks are organized in clusters with one central gateway node per cluster. This **gateway node** is not or at least less energy constrained than the sensor nodes and can therefore manage network communication and routing based on energy states of the other nodes.

Because systems like the ones mentioned until now are only supplied energy by a battery, they eventually run out of energy and therefore stop working. One approach to avoid this, is to supply the nodes with a constant stream of energy through an external source. As laying cables to each node is obviously infeasible and usually even impossible due to the network distribution and sensor locations, better options had to be found. One option explored in multiple papers is called "Wireless Power Transfer" (WPT), where there are special energy transmitting nodes as part of the network that supply power wirelessly to the sensors [73].

Another possibility to get energy from an external source is harvesting energy from the environment. One of the first papers to propose this for WSNs was [34]. For energy harvesting in general, there are countless different methods. Some use easily accessible environmental features, like solar energy [57], which seems to be the most widely used one, wind energy [71] and thermal energy [40]. Other works cover lesser known possibilities, like electrochemical energy harvested from bacteria [64] and using vibrations that are present in the environment (e.g. waves) [72]. Shaikh and Zeadally [63] sort energy harvesting schemes for WSNs into 3 categories. There are methods that harvest energy from the ambient environment, like solar energy and wind energy, external mechanical sources like vibration sampling and external human sources that use the energy produced through an activity conducted by a human, for example. The applicability of an energy harvesting scheme to a particular WSN is usually heavily influenced by the type of WSN that is used.

The authors of [34], despite their paper being one of the first on this topic, already address the main problem of using energy harvesting, which is that the incoming energy is usually neither constant, nor easily predictable. They however conclude that the system has to use some form of prediction to adapt their operation to the incoming energy. In order to get such predictions, they aim to learn harvesting profiles based on past patterns. For this purpose, they introduce the Environmental Energy Harvesting Framework (EEHF). It consists of multiple different parts, spectral estimation, which detects patterns in the energy harvesting values, the prediction filter, an energy consumption predictor and a component that combines the values received from the other components into a cost metric. The computed output can then finally also be transmitted into the network to better facilitate global decisions like routing and load balancing. Overall, the introduced framework can then be used to align the nodes operation with the predicted available energy and therefore extend system operation time. More concrete algorithms and approaches are also presented in literature that aim to predict future energy harvesting profiles. One frequently used option is using Exponentially Weighted Moving Average (EWMA), which calculates a weighted sum of the last real measured value and the last predicted value to form the new prediction [13, 32]. Weather Conditioned Moving Average (WCMA) is another algorithm, specifically used for predicting solar irradiance, which was based on EWMA, but also takes into account how weather may affect future harvesting values [54]. The authors of [7] compare these two algorithms to another predictor developed by the Swiss Federal Institute of Technology of Zurich (ETHZ) [51, 52] that calculates N predictions for future harvesting values from predictions of the past. They conclude that for prediction of solar energy harvesting, WMCA is by far the most accurate, while EWMA and ETHZ are quite similar. EWMA, however, is much faster than WCMA, making it valuable in certain situations as well. As different methods of energy harvesting have varying predictability, Kansal et al. provide a classification of energy harvesting sources based on their predictability and controllability. These factors can also be very influential in the decision, which form of energy harvesting to use for a certain system. The full class definition including explanations and examples is summarized in Table 2.2. The most prevalent energy harvesting method investigated in literature for WSNs is solar energy, due to its relatively accurate predictability using weather forecasts and sun position, it following a known day-night cycle and it being easily harvestable.

Regarding concrete design of wireless systems that use energy harvesting, Raghunathan et al. [57] explored multiple options and considerations for every part of the system. Like many papers, they mainly focus on solar energy and start by exploring characteristics of solar cells. It is stated, that a solar cell, in contrast to a battery, does not act as a voltage source, but is a voltage limiting source. Moreover, the circuit voltage stays about constant, even if the solar irradiation and thus the current decreases or increases. Finally, the fact that a solar cell has an optimal operating point is also considered. Regarding energy storage, which is still necessary to supply the node

Category	Examples and Characteristics
Uncontrolled, predictable	Solar energy; availability follows night/day and seasonal cycle
Uncontrolled, unpredictable	Vibrations; irregular occurrence.
Fully controllable	Human input; through active work
Partially controllable	RF power; can use transmitters, but exact energy cannot be controlled

Table 2.2: Classification of Energy Sources by Controllability and Predictability

with energy in times of low harvest and can be charged during high harvest phases, they compare batteries and ultracapacitors. While batteries generally offer a higher energy storage capacity in relation to their size, ultracapacitors are more power-dense and can handle power spikes better. In addition, they also offer a longer lifetime in terms of charge-discharge cycles, which can be a factor limiting the operating time of an energy harvesting powered system. Batteries on the other hand, allow less energy leakage than ultracapacitors. When designing the harvesting circuit, they take the optimal operating point of the harvesting source into account and try to design the circuit around it, so the circuit power stays at the optimal level. To be able to base decisions on the amount of harvested energy and level of battery charge, the incoming power and stored charge have to be measured. Low power battery monitor Integrated Circuits are proposed for this purpose. Finally, they demonstrate a real implementation of their proposed system, that operates with harvesting aware power management, named Heliomote. A more recent example for a design of such a system can be found in [37]. The authors develop a Wireless Sensor Node that harvests Radio Frequency (RF) energy. In contrast to the Heliomote, they choose an ultracapacitor as energy storage with the load and the switch being optimized for minimizing energy leakage. Different options for each of these design features are evaluated and a system is proposed, that is cheap, easy to produce and integrate, but has a high loss factor and a limiting dielectric constant (electric polarizability in relation to that of a vacuum).

While generally providing a good and easily accessible energy source, energy harvesting also has some downsides and introduces new problems in WSNs. As already stated, the sensors often have to rely on harvesting predictions, that have to be of a certain accuracy, since mispredictions can cause the system to run out of energy and even lose recorded data, if the actually received energy is less than predicted [66]. They also cause many new design challenges, for harvesting itself as well as for energy storage, as the supply voltage of the harvesting source is usually variable [57]. The variable nature of most energy harvesting sources also introduces the need for very complex scheduling algorithms, that try to avoid the system running out of en-

ergy, while maximizing performance [68].

The easiest solution for this, the idea of matching the energy consumption to the amount of harvested energy, was presented first in [34] and was extended in subsequent papers. While the first paper only implies how to use knowledge or predictions of incoming energy (e.g. for routing), later research papers build a more comprehensive model on which they base concrete approaches. In [33] they state the idea to have a singular node run for an unlimited time by constantly running the node with the average power that is supplied by energy harvesting at a certain time. This works under the assumption that the energy storage is big enough, so excess energy harvested in high-energy phases can be stored for times of low harvest.

Kansal et al. [31, 26, 32], later introduce an idea for Energy Neutral Operation (ENO), that is used in many other papers on energy management schemes and will be used in this thesis as well. The authors first state that an energy harvesting system can theoretically have an unlimited operation time, if it is guaranteed, that the system has at least the amount of battery charge at the end of a certain time span, as at the start and the battery charge never falls below a predefined minimum (defined as 0 in this paper) during the span. The authors also extend this model by proposing to divide up the time span into discrete time slots and assigning an operation level to each time slot, that consumes a set amount of energy. Using this approach, it is possible to match the operation levels of a node to the energy predicted to be harvested in each time slot, such that ENO is guaranteed and the node performance is maximized, by solving an optimization problem. They also account for having to dynamically adapt the operation level in case of inaccurate predictions. This approach yields significantly better node performance compared to assigning one constant operation level, given the battery does not have perfect efficiency, as more energy can be directly used without being stored first. A drawback of this simple approach is that the node has to solve an optimization problem at the start of every time span. The general idea is taken further in the paper [26], where the authors propose a concrete, low complexity approach for the initial assignment of operation levels based on the same model, which also still provides almost optimal results in terms of node performance. Finally, in [32] this approach is made applicable to a larger amount of node design possibilities. They account for systems without energy storage, with perfect and non ideal storage, that has passive energy discharge. Furthermore, different characteristics of energy harvesting possibilities are considered that are shown in Table 2.2.

In all of these papers, the technique to scale node performance that was chosen is Duty Cycling. When using this technique, the energy consuming device switches between an active and a sleep state in a periodic cycle. The duty cycle itself is defined by the fraction of time the device spends in the active state during one complete cycle. A 20% duty cycle for example would therefore mean, that the device on average over the complete cycle spends 20% of time performing its work and 80% of time inactive. Using this approach can significantly reduce the amount of energy a device consumes, while it usually does not impact performance too much, providing

a high enough duty cycle was chosen. An example for this is a temperature measuring device, which takes one second to take a measurement and for which it may be satisfactory to take a measurement every 20 seconds. Here you could choose a duty cycle of 5% and, assuming that the energy consumed by the device in the sleep state is low enough to be negligible, the energy consumed would be reduced by 95% compared to if the device stayed active all the time. Even if the energy drawn in sleep state is non negligible, but just less than in active mode, it offers the possibility of greatly reducing energy consumption. The authors of these papers consider a minimum duty cycle D_{min} , so a minimum performance is guaranteed and the node cannot shut down completely, as well as a maximum duty cycle D_{max} . The reason one would want to increase the duty cycle, is that a higher duty cycle yields a higher system performance, also referred to as utility in [31, 26, 32]. This concept can be explained by the temperature sensor example as well, since one would get a more accurate and significant picture of the environment by increasing the frequency of measurements, thus increasing the duty cycle. Another advantage of using duty cycles in a model, is that one does not have to consider different task functionalities (e.g. measure, transmit, save), as all tasks that have to be executed are included in the chosen duty cycle. In conclusion, [31, 26, 32] all target the same optimization problem, [31] choosing to solve it directly, while [26, 32] provide an approximate solution with lower complexity.

The general objective of the presented optimization problem is to maximize the summed duty cycles over all time slots and therefore also the system performance or utility, while staying energy neutral, as explained out before. The duty cycle at all times has to stay between the maximum and minimum. The battery charge in each time slot is calculated by using the battery state from the previous time slot and subtracting the energy that is drawn from the battery, if the consumed amount of energy is higher than the harvested energy. Finally, it adds the amount of energy that is stored in the battery while the system is in sleep mode and the amount of energy stored while the system is active, if it uses less energy than is harvested. The battery charge in this model is not limited to a maximum and a constraint requiring the battery to never drop below a minimum is also not given, but it is not needed in this case, because requiring a minimum operation level forces the system to provide continuous energy, thereby enforcing an implicit minimum battery charge.

There are also different ways to achieve energy neutrality in energy harvesting wireless systems, one of them is to calculate an energy budget for a time span and distribute the energy consumption accordingly [60], or trying to match the energy consumption to the energy harvested, which can make an energy storage unnecessary [44]. The downside of these approaches is that it does not guarantee unlimited operation, the latter one would only have this problem when using an energy source that has phases where no energy is harvested, such as solar energy.

Along with these papers, there are many other publications about trying to maximize system performance while staying energy neutral in energy harvesting wireless sen-

sensor nodes. The understanding of system performance and how it is scaled in these papers varies greatly and will be investigated in the following paragraph. System performance, often also being referred to as Quality of Service (QoS) is a metric of WSNs, that, similarly to energy management, was first explored in routing. For this specific application, the most commonly mentioned QoS metrics are delay and reliability, which usually scale with the amount of energy that has to be used [1, 25, 20]. Another perspective on QoS in a network setting is provided in [28], where the number of sensors communicating data to a base station is considered the targeted QoS metric.

Looking at QoS from the perspective of a singular node, duty cycles and their utility are, as mentioned, one possible interpretation as a metric. The definition for the term utility used in [23, 22] is another possible QoS definition, which is dependent on the time an action is performed. An example for this is that measuring the change in soil moisture is more useful in wet seasons than in dry seasons. Thus, in order to model the time-varying utility, a utility profile, like in [23], can be used, that is based on the maximum performance and the current performance.

Explanations other than Duty Cycling and utility are also prevalent, for example, just using different sampling rates, which is similar to the Duty Cycling idea, but limited to the task of taking measurements only, using different available hardware components, like transducers with varying energy consumption and having different communication patterns (e.g. either instantly sending or grouping packages) [15]. According to [60], varying QoS may also occur as a result of using different algorithms to solve a problem, or using the same algorithm, but with different input parameters, which both results in having to perform varying amounts of computation, therefore consuming a different amount of energy and having a varying accuracy of the results of the calculation.

The QoS is displayed as a numeric value in all papers and is positively correlated with energy consumption. However, the type of function describing the correlation may differ. It can be a linear relationship, as, for example, in case of in the optimization problem from [31] explained above, using the duty cycle as a direct metric. Alternatively there is also the possibility of having a concave function [48], or QoS being a more abstract value, not described by a function [9]. Finally, to conclude this part about QoS, it should be mentioned that the different definitions and representations of QoS are often connected to the specific application of the WSN, that can have differing sensing goals. A WSN recording temperature data has to focus less on the transmission delay, for example, than a WSN deployed in a battlefield. A summary of the different ways to scale node performance and the related QoS definition is presented in Table 5.1

The general idea in energy management schemes for energy harvesting wireless sensor nodes is to assign different workloads to different times in order to assure the node operates for as long as possible and does not run out of energy. An example for such a scheme was already shown in [31, 26, 32], where different duty cycles

Method	Work(s)	Explanation	QoS relation
duty cycle/task rate	[31, 26, 32, 15]	how much time is spent executing a task/task rate	e.g. more measurements
Voltage Scaling	[65]	change supply voltage influencing performance & consumption	same as duty cycling
different Hardware	[60]	varying hardware with differing energy requirements	e.g. measuring accuracy
different Software	[15, 60]	algorithms with varying complexity/different logic	e.g. calculation accuracy

Table 2.3: Concrete methods to scale energy consumption and system performance

are assigned to time slots of equal length. The general idea of time slots is used in many papers, however, the "unit of work" (for example the duty cycles) often varies between different papers. Some papers refer to the units of work as "tasks".

Caruso et al. for example [9] assign tasks to the discrete time slots, which can have varying energy consumptions over the time slot and QoS values. In this model, the application has a fixed set of tasks with a unique QoS and energy consumption value, for which any functionality can be completely disregarded when finding the assignment. The same task definition will be used in this thesis.

The term "task" also appears in other publications, such as [17]. Here, the tasks have differing functionalities and there is only one version of tasks for every desired functionality. Additionally, a task is **defined its** execution time and by the states it puts the systems components into, which can be used to derive the energy used by it. Moreover the tasks themselves are not assigned directly, but grouped into so called "Applications" that cover every functionality and are executed periodically. By using a varying duty cycle, different versions of the application can be created, with all of them covering the same functionalities with varying energy consumption and QoS as a result. These applications can then be assigned to any point in time, as the presented model does not use the time slot framework introduced in [31]. In their later work [15], the same authors present yet another model, this time choosing to use the time slot framework. Tasks, like in their other model, have a certain functionality, they are however modeled differently, having an execution time, period and energy consumption and multiple task variants exist for a single functionality. These avail-

able tasks are grouped into task sets that include one task for each functionality the system should have. From these, execution plans can be derived, that cyclically execute all the tasks in the set and, as a result of the varying task versions, have differing energy consumption and QoS. The execution plans are then assigned to the discrete time slots. The authors of [66] choose to use a similar idea, calling the unit of assignment "configurations" instead of execution plans. Escolar et al. extend this task model even further in [16], where they consider a sensor network model with a sink node, that receives the data sent by the sensors. Consequently, the send and receive tasks executed by sensor and sink have to be somewhat aligned in their period and execution time, introducing the concept of "task compatibility".

A task in the real-time model used by Moser et al. [45, 47, 46] is also characterized by a specific energy consumption and is finished as soon as the energy requirement is met. It hence does not have a defined execution time. The time it takes only depends on how much energy the system provides. Tasks also have a deadline. They need to finish before it to satisfy the real-time requirement of the model used. Finally, tasks may be interrupted by other tasks and resumed at a later time. In their later work [49], another model is used, based on the discrete time framework. Instead of duty cycles, in this paper they assign more abstract Service Levels to the time slots. A specific Service Level is characterized by a reward (QoS) and an energy consumption value. This model can be realized in a multitude of ways, for example with different services having the same tasks but with varying duty cycles or by them having different task versions. The general idea is furthered in [48], where continuous service levels are also accounted for. A continuous service level also has a specific energy consumption and its reward can be calculated using a concave function with the energy requirement as input.

In [60] there are also real-time tasks that have multiple versions with specific execution time, energy requirement and reward. Tasks can be frame-based (all tasks have a common period) or periodic (different periods) in this model, moreover, they can be mandatory to be executed at least once per frame/hyperperiod, or optional. Tasks have a deadline, which usually is their period, so all frame-based tasks share the same deadline. Finally, versions of tasks with lower reward are always executed faster and consume less energy than versions with higher reward. The scheduler can then decide which version of a task to assign to a frame/hyperperiod, based on how much energy is available.

Another completely different model is considered in [22] where they aim to compute a target State of Charge (SoC) of the battery for the next day based on energy harvesting predictions for the span of one year and determine a suitable duty cycle for the controller to execute the tasks at, so the targeted SoC is achieved. The utility in this model is, as already stated not only dependent on the assigned duty cycle, but also on the point in time.

As the phrase "task" occurs often in the literature mentioned above with varying meanings and properties, Table 2.4 aims to provide a general overview of the dif-

ferent task definitions, trying to assess the abstraction level of the concept described as "task" as a result. Now, looking at the different model definitions in these publi-

Work(s)	def. Functionality	Grouping	Execution Time	Deadline	Abstraction Level
[9], this thesis	✗	✗	✗	✗	High
[17]	✓	✓, Applications	✗	✗	Medium
[66]	✓	✓, Configurations	✗	✗	Medium
[15]	✓	✓, Execution Plans	✓	✗	Medium
[45, 47, 46]	✓	✗	✗	✓	Low
[49, 48]	✓	✓, Service Levels	✗	✗	High
[60]	✓	✗	✓	✓	Low
[22, 23]	✓	✗	✗	✗	High

Table 2.4: Definitions of the term "task" across various publications

cations, it is important to understand, that they solve specific subtypes of the before explained challenge of energy budget management. Therefore in the following, the varying problems and their solution approaches will be explored.

Tackling the problem of energy management in rechargeable real-time systems, which is also applicable to WSNs, Rusu et al. [60] propose a multi-version scheduling framework and a heuristic algorithm that determines how tasks should be executed during a battery discharging phase. The goal is to decide, for every time frame or hyperperiod, which version of each task should run and at what speed so that the total "reward" (QoS) is maximized, while still meeting both a global deadline and a strict energy budget that comes from the energy accumulated during the charging phase before and is distributed across the frames using another heuristic. Usually, exactly one version of every task must be scheduled in each time frame. However, the model can also account for optional tasks by introducing a special "dummy" version with zero execution time, zero energy consumption, and zero reward. Because the algorithm only uses the stored energy that was harvested the charging phase, it does not rely on future energy harvesting and does therefore not have the problem of inaccurate predictions. The basic idea of the developed algorithm is to assign the lowest version (reward and speed wise) of all tasks and then upgrade them to meet the deadline and to get a higher reward.

Another different energy management problem and approach that also targets real-time sensor nodes, is supplied by Moser et al. in [45]. They use, as stated before, a task model with arrival times and deadlines, with the tasks execution time being dependent on the power that is supplied by the node. Their goal is to find a schedule for the arriving tasks, which violates no deadline, while working under a limited energy budget. For this purpose, they develop a specific scheduling approach called Lazy Scheduling Algorithm (LSA), to avoid deadline violations which would occur

for other algorithms like the As Late As Possible (ALAP) policy. LSA also needs predictions of harvested energy as an input.

The same authors moreover also consider a task reward (QoS) in [50], changing the goal from meeting deadlines to maximizing reward while managing the available energy. They want to assign "services" containing tasks to a fixed amount of time slots, these services can be supplied with a specific amount of energy, which also determines the gained reward. To further clarify, the energy that can be supplied to a service and thereby also the reward are continuous in this model, the reward is moreover a strictly concave function of the supplied energy. The goal for the assignment, similar to Kansal et al. [31] and Caruso et al. [9], is for it to result in an energy neutral schedule that maximizes the summed up reward. As solutions, a greedy algorithm is presented first that is later refined into a less limited algorithm called Recursive Decomposition (RD). For the specific problem and model used, this algorithm supplies an optimal solution. To extend this model for discrete reward levels energy requirements of services, they firstly adapt the greedy algorithm and RD algorithm to the changed model in [49]. However, as seen during evaluation, the quality of the solution is heavily impacted by this change. Therefore, two more algorithms are developed. The first one, called "Search" in the paper, starts by applying the lowest service level to all time slots and then searching all possible feasible solutions that have non-decreasing energy consumption over the course of the time span, while assuming there is enough storage capacity to store the entire energy harvested in the next time span. The second proposed approach is a Dynamic Programming algorithm, which can additionally receive a parameter determining how large the search space and how close to the optimal the final assignment will be.

Kansal et al., as already explained, aim to maximize the summed up duty cycle (or and associated utility metric) over a time span split up into discrete time slots, while having at least the same battery charge at the end of the time span as at the start. Across the different papers released by the authors on this problem, they present differing solution algorithms. The simplest approach is presented in the first paper in this series, [31], directly solving the laid out optimization problem and then dynamically adapting the planned duty cycles over the course of the time span if the actual harvesting values differ from the predictions. This approach, in addition to maximizing the utility of the system, also minimizes energy loss in storage, as it leads to the system using most of the energy harvested directly and has a low probability of complete energy depletion, as D_{min} is chosen during low harvest. Therefore, the battery charge would be used at the slowest possible rate. In case the actual harvesting values are not equal to the predicted ones, a recalculation method is presented. It is conducted by iteratively reducing the duty cycles to D_{min} in times of low energy harvest, if the actual budget is smaller than the schedule planned for, or scaling up the duty cycle to D_{max} during high harvesting yields, if there is a higher charge than planned. The approach demonstrated in [26] first assigns D_{min} to all time slots and then greedily increases their duty cycle, based on how much energy is produced during the slot,

also having a separate assignment logic for slots that supply enough energy to sustain node operations and slots that do not. Finally, in [32], D_{max} is assigned to all time slots that sustain the node's operation through energy harvesting and D_{min} is assigned otherwise. If the available energy according to the predictions is not used fully, D_{max} is assigned to the highest harvesting slots in the non-self sustaining slot set, **until there is no more energy is left to use**, then the remaining energy is assigned to the next highest harvesting slot as a duty cycle between D_{max} and D_{min} . In case the system does not receive enough energy to get throughout the initial schedule, it calculates the distance of the overall schedule to being energy neutral and distributes the necessary consumption decrease equally over the self-sustaining slots. The last two papers present the same idea for recalculation during the time span in case of deviating harvesting values, increasing the duty cycle assigned to non-self-sustaining slots in case of more energy than predicted and decreasing the duty cycle of self-sustaining slots to D_{min} if the energy values are below the target.

Caruso et al. choose to use a similar framework as Kansal et al. for their problem, however, instead assign abstract, functionality independent tasks to the time slots, more precisely, exactly one task per time slot. These tasks also have a unique energy consumption and QoS, similar to a specific duty cycle. The presented problem in this paper is also similar to Kansal et al., with the only differences being that there is a minimum and maximum battery charge, instead of the duty cycle limitations to assure constant operation and all energy values being integers. The battery charge should never be below the minimum B_{min} between time slots and cannot exceed B_{max} . They choose to represent their problem as an Integer Linear Program (ILP), for which they present a Dynamic Programming (DP) algorithm as a solution. This solver uses energy harvesting predictions and iterates over all possible system states (characterized by a specific battery charge and time slot), linking them with the available tasks. Describing it with more detail, they have two arrays of size (possible battery levels $\times$ time slots), one in which they save the best possible summed up quality values of tasks assigned that lead to an energy neutral end state, in the other one they save the task that lead to the best next state. To get the solution, **one has retrace** the path of the tasks in the array that saves the tasks. This approach always finds the best possible solution quality wise, that is energy neutral **(providing the linear program is solvable)**. Reassignment of tasks in case of diverging harvesting values can be conducted by using the solver only for the remaining time slots and finding the tasks from the state the system is currently in to the last time slot. The reassignment can also be computed with new harvesting values. Overall, while it does find an optimal solution, the approach itself is costly in terms of time and memory, which may pose a challenge in resource constrained WSN nodes. In their later paper [10] they extend their problem statement by adding another optimization goal, which is called "stability". The general motive of this paper is, that too many changes in the QoS of tasks assigned in succession may be a problem, as it makes data difficult to manage and process. Therefore, they try to find a tradeoff in stability and quality using

their dynamic programming algorithm by penalizing quality levels of tasks based on how large the leap of quality is compared to their predecessor. This achieves a non-optimal solution quality-wise, but may be very useful for specific use cases.

A simpler, approximate solver is introduced by [15]. The underlying model is similar to the one used in [9], using "execution plans" instead of tasks as unit of assignment. The general idea is to sort the available execution plans by their efficiency, calculated by dividing their QoS by the energy consumption in a time slot, and assigning the most efficient plan to all slots first. The algorithm now checks for energy neutrality and the battery charge falling below the minimum. If any restriction is violated, the slots in lower energy harvesting phases get assigned execution plans with lower energy consumption and QoS, until both restrictions are satisfied. After this, or if the initial assignment is already energy neutral and the minimum is never exceeded, the algorithm may upgrade some time slots in phases of high harvest to less efficient, higher quality execution plans, until it is not possible anymore due to one of the two energy restrictions. A separate algorithm for recalculation of the remaining slots in case of inaccurate predictions is provided alongside, keeping the general idea of "upgrading" the execution plans in high energy harvesting slots if more energy is available and "downgrading" them if necessary due to lower harvest. This approach, referred to as "Upgrade/Downgrade strategy", is much more lightweight than the DP algorithm in terms of execution time and memory usage, but does not always find the optimal solution.

The authors of [66] provide another algorithm comparable to the last two, which, in contrast to the algorithms shown before, does not calculate one solution at the beginning of the time span, but assigns the operation configuration for every time slot at the start of the slot. The general idea is to assign the configuration of tasks with the highest quality to the next time slot, that, if kept constant until the last time slot, still results in an energy neutral solution. For this calculation, only the current battery charge is considered, while future energy harvesting values are completely disregarded, meaning there is no reliance on possibly inaccurate predictions and making a recalculation, as in the other algorithms, not necessary. The resulting schedule is guaranteed to be energy neutral, the algorithm is computationally even simpler than the approaches mentioned before and does not need any consideration for unreliable predictions.

The final approach investigated in this section is the State of Charge (SoC) based algorithm developed by Hanschke and Renner in [23]. The model used also considers time-varying utility, meaning the QoS of the node can change at different times despite constant performance. The first step is to predict the SoC over the next time span and identify the extrema in the prediction, in case of solar energy harvesting, for example, day and night, thereby dividing the time span into two regions. For each of these regions a specific power value is then calculated, by finding the maximum power consumption in the region, that does not deplete the energy budget available. Due to how utility is defined in the paper, it provides the upper bound for the energy

available in a time slot. Finally, based on the distribution of energy achieved in the steps before, a schedule can be generated, using duty cycles as an example, but the exact method to do it is not specified.

To conclude this part of the literature review, Figure 2.1 provides an overview of all models and their solution approaches, linking those that are comparable to each other.

2.2 Graph-based Path Finding Algorithms

As the goal of this thesis is to investigate the energy budget management problem in Wireless Sensor Nodes using a graph-based model that transforms the task of finding a quality-wise good and energy neutral schedule into a path finding problem, the following section will be dedicated to reviewing such algorithms. The explanations of the algorithms here assume that the goal is to find the shortest path, but transforming the algorithm toward using another path optimization goal is usually very easy. To start, there are algorithms like Depth First Search (DFS) and Breadth First Search (BFS) [19, 43], that can be used to find any path from a source node in a graph to a specific target node, while having a time complexity of $O(|V| + |E|)$, with $|V|$ being the amount of nodes/vertices in the graph and $|E|$ the edges. For both algorithms, the path from each node back to the source node can be retraced, if every visited node memorizes the neighbor it was discovered from. Then the desired path can be found by following these references from the goal to the source. It has to be noted, that these algorithms do not necessarily find an optimal or even "good" path.

For this exact purpose, there are other well known methods, like Dijkstra's algorithm [14], the Bellman-Ford algorithm [6] and A* [24]. The former is a greedy algorithm, finding the best paths from a designated source node to all other (indirectly) connected nodes in the graph. It saves the shortest found distance for each node, initialized to zero for the source and to infinity for all other nodes at the beginning. It also maintains a priority queue of nodes ordered by their distances of the best path found to them from the source at the exact point in the algorithm. Repeatedly, the algorithm extracts the node with minimum distance, declares its distance final, and reduces the distances of all neighbors of this node, if a shorter path is found via the extracted node (also called relaxing the edges). The algorithm terminates when all nodes have been processed. Overall, its running time, depending on the data structure chosen for the priority queue can range from $O(|V|^2)$ to $O(|E| + |V|\log|V|)$. The Bellman-Ford algorithm uses an exhaustive approach for the same problem, checking for all edges, if they can be used to improve the path to the node the edge goes to, repeating this $|V| - 1$ times. This results in a time complexity of $O(|V||E|)$, however, other than Dijkstra's algorithm, it is able to handle edges with negative weights in the graph. The A* algorithm can find a solution for the problem of finding the best path to a certain node even faster, if it is given a good heuristic of judging the distance of any node in the graph towards the goal, with it **being at least fast as** the best-case

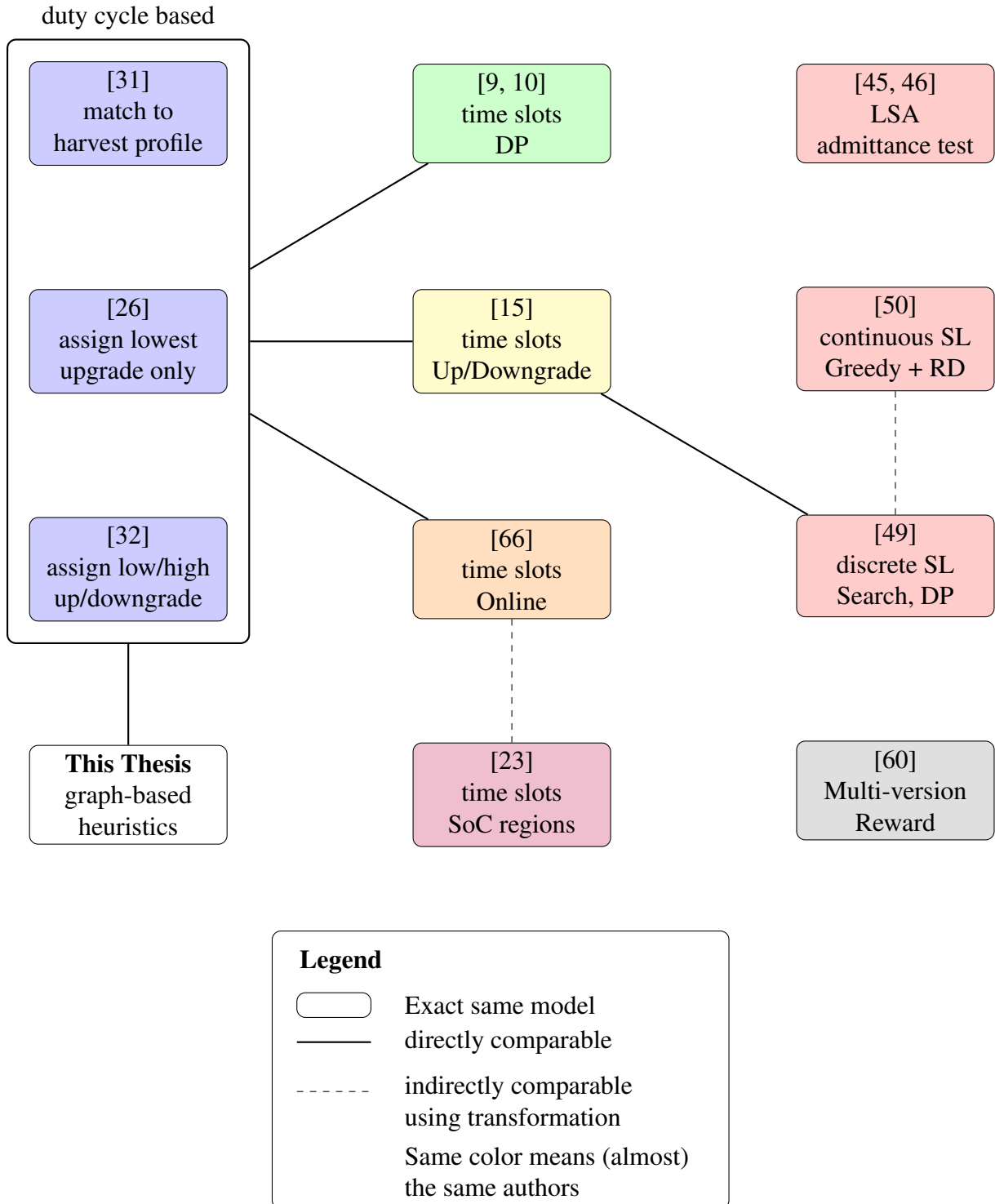


Figure 2.1: Relationships between EH-based scheduling models applicable to WSNs

of Dijkstra's algorithm [53]. It maintains a priority value for each discovered node based on the node's distance from the source and an approximate distance to the goal, that is given by the aforementioned heuristic. The algorithm repeatedly expands the node with the smallest (best) value, reducing the priority values for all neighbors, if possible through this node and updating the neighbors' priorities in the priority queue. If the used heuristic is admissible (never overestimates), A* is guaranteed to return an optimal path.

Since the graph model used in this thesis will later be shown to always be a Directed Acyclic Graph (DAG) it additionally makes sense to review algorithms and heuristics specifically aimed at finding the best paths in DAGs. A DAG in general is defined as a graph, which has edges that are described by ordered pairs of nodes $(u, v) \neq (v, u)$, thereby declaring a clear direction, and does not contain cycles [70]. A standard approach for the shortest/best path problem in DAGs is to use topological sorting on the graph [29], which provides a topological ordering of the nodes in the graph, meaning, that for every edge (u, v) in the graph, node u comes before v in the ordering. This order can only be found for DAGs and is not necessarily unique, as nodes that are independent of each other can be on the same topological "level" and therefore are exchangeable in the final ordering. Multiple ways exist to find the topological ordering of a DAG, one of them being based on DFS, as described in [69]. In order to gain the best path to a goal node, **on has to merely** save the currently shortest path, that was found to a node, in the node and update it if a shorter path is found, when the node is being found again through another neighbor. Alternatively, only the best paths value and the best parent (neighbor with edge pointing to the node) can be saved and the best path retraced backwards at the end. Overall, this results in a time complexity of $O(|V| + |E|)$, better than all approaches that find an optimal solution shown in the section before.

Another way to decrease the time complexity for the solution of the path finding problem in graphs is to use a heuristic. By accepting a possibly non-optimal, but still good solution, it might be possible to radically reduce the search space. A fitting heuristic, that does exactly that, is Beam Search, an approximate search algorithm that was originally used in the Harpy Speech Recognition [39]. The procedure to build the search tree is generally based on BFS, however, the algorithm receives an additional parameter called "beam width" β . This parameter determines, how many neighbors of each node get further explored as part of the next level of BFS. Starting from the root, the algorithm generates all successors of the current node, evaluates them using a scoring function (for example edge costs), and then keeps only the best β nodes while discarding the rest. It then proceeds to the next level and repeats this process for all nodes chosen in the previous iteration, effectively iterating over increasing depths of the graph while maintaining only a narrow "beam" of promising candidates. This pruning massively reduces the search space and memory requirements compared to exhaustive methods, making beam search efficient for large problems, while sacrificing completeness and optimality because potentially good paths may be removed

early if they do not appear promising according to the evaluation function. The time complexity of standard Beam Search was proven to be $O(n_{max}k|V|\log(k))$, with n_{max} being the maximum sequence length (how many steps are taken until a solution is found), k is the beam width and $|V|$ represents the number of nodes created during beam search (not number of nodes in the graph) [41].

Random sampling is a method that can be used to randomly find or approximate solutions to various problems. One of the first descriptions of this was developed by Stanisław Ulam, called Monte Carlo method [42]. The general pattern applications of the Monte Carlo methods have to follow, is that there have to be a defined set of inputs, from which a set amount of inputs are randomly chosen and used. The output generated from these inputs can then be analyzed and used to approximate a solution. The most famous example for this is the approximation of π , whereby a quarter of a circle is put into a square with known area, with the circle having the same radius as one side of the square. In this square, points are randomly chosen, after that, the ratio of points inside the circle can be used to estimate π by calculating $\pi \approx \frac{4r}{n}$ (r being the amount of points in the quarter circle, n the number of samples). Obviously, for this to work, a high enough sample number has to be chosen and the samples have to be uniformly distributed. A method to apply this to finding good nodes and paths in trees is supplied by Monte Carlo Tree Search (MCTS), an idea formally developed and described for the first time by Rémi Coulom [12]. The heuristic moreover has multiple use cases, the most common one being finding the best possible decision in a specific situation, as applied in Go or Chess for example [8]. One edge therefore represents a decision and a node is a state of the system. The general procedure can be split up into 4 phases which are repeated for a amount of iterations chosen by the programmer, displayed in Figure 2.2. In the basic version the tree is built during execution, with one node being added per iteration.

It begins with the Selection phase, starting at the root by repeatedly selecting the best child node, according to a formula based on a node's value from previous iterations and the amounts of visits to a node. This is repeated until a node is found where at least one child can be added. The algorithm then enters the second phase, called expansion. If there is more than one child to add, it does not matter which child is added, it can be random. Outgoing from the newly added node, a lightweight simulation is conducted, using random moves until an end state is reached. Note, that no new nodes or edges are added in this phase, which makes the simulation "lightweight", only the necessary system state values affected by the random moves are saved. The result of the simulation (e.g. 0: loss, 1: win) is then backpropagated along the path taken in the Selection and Expansion phase, updating their node values and visit counter. After this, a new iteration can begin. After finishing the set number of iteration, usually supplied before execution, the best next move(s) can be derived based on the value the formula determining the best children returns. The resulting tree can also be used for the purpose of path finding if, for example, the best path found in an iteration is always saved or the best node path is followed toward a leaf node with the desired

characteristics (provided one was created during execution).

As for the formula used in the Selection phase, the most mentioned and used one is called Upper Confidence Tree (UCT), taking the node value w_i , the visit counter of node i n_i , and the node's parent's visit counter N_i as an input and calculating its output as follows:

$$\frac{w_i}{n_i} + c \sqrt{\frac{\ln N_i}{n_i}} \quad (2.1)$$

c being the so called "exploration parameter", often being $\sqrt{2}$. The first part is just the average result of simulations outgoing from this node, the second part assures, that the heuristic does not always exploit the same nodes and path, but also conducts some exploration among less visited notes.

In order to adapt this algorithm to DAGs, the authors of [5] and [61] present some changes to the iteration phases, that make it possible and improve the result's quality. Similarly to this thesis, they also only consider DAGs with one defined root node (rooted DAG). They describe a rooted DAG as a tree, in which all nodes that represent the same value on one level are collapsed, forming so called transpositions. In the selection phase they propose the use of the Upper Confidence Bound for rooted Directed Acyclic Graphs, which takes into account that transpositions may have multiple parents. Moreover, it is proposed to store the simulation results and visit counters in the edges rather than in the nodes, as the node no longer completely represents the result of one specific move. In the **backpropagation** phase, the path for backpropagation is also no longer obvious, since one node may have several parents. Therefore, multiple ways to conduct the backpropagation are shown. They include iteratively updating every ancestor of the node the simulation was started at, updating only the nodes on the path chosen in the selection phase and updating the selection path while modifying the aggregation of all other ancestors in a different way.

Another view on MCTS that might be interesting for this thesis is mentioned in [11], where the idea of parallel execution of multiple simulations, instead of just one, is mentioned. It can lead to increased efficiency as there are more "random samples" without having to go through all other iteration phases more often, increasing the amount of information one iteration yields.

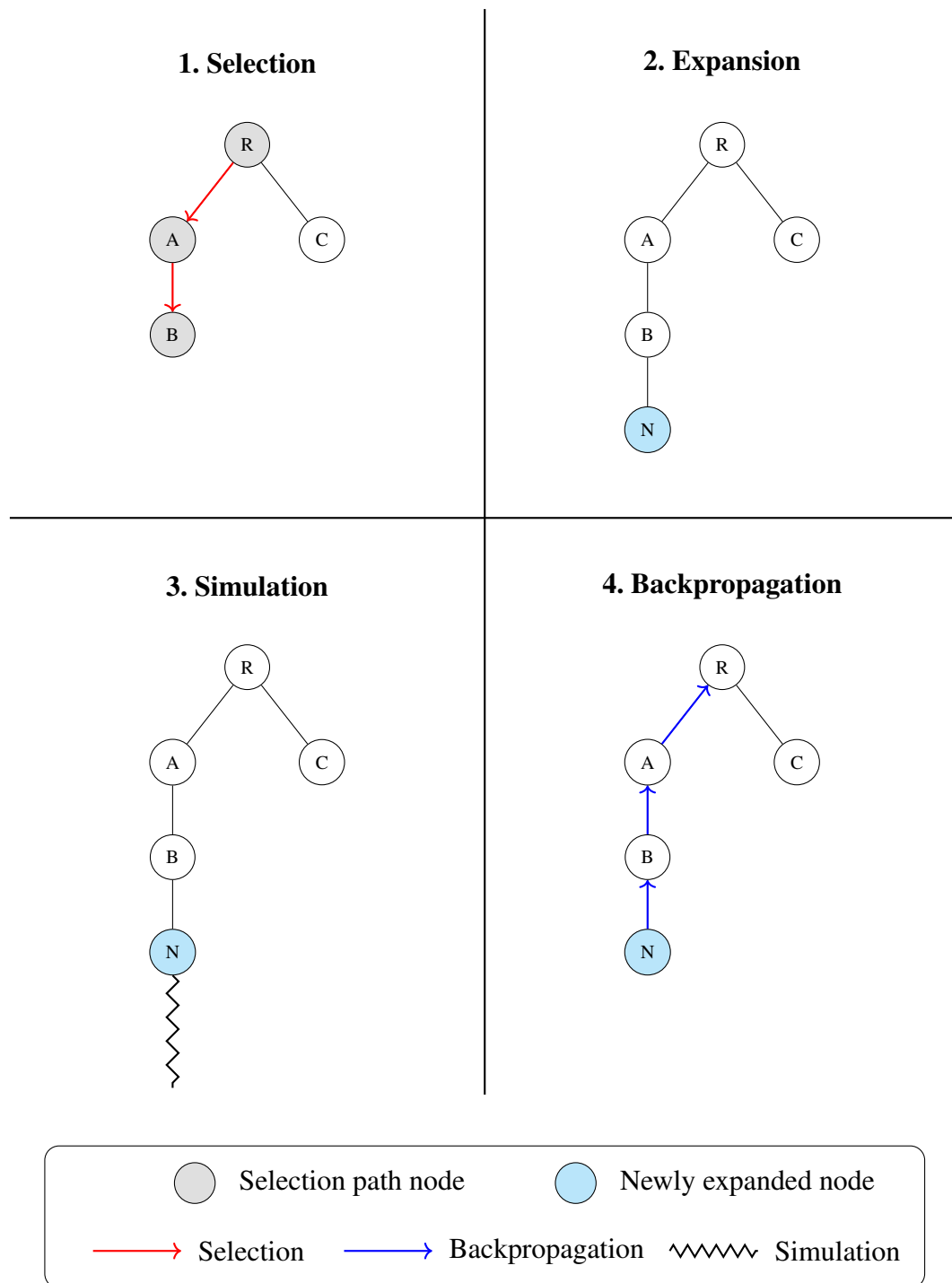


Figure 2.2: Monte Carlo Tree Search phases

3 Fundamentals

As a base for the upcoming explanation of the developed algorithms for the energy budget management problem in Wireless Sensor Nodes, the following section will revolve around explaining the model, that served as a base for the development of graph model, the solvers and for the simulation.

3.1 System Model

In general, a Wireless Sensor Node consists of a sensing unit to take measurements, a processing unit to process the measurements and a (flash) storage unit to save the resulting data in. Additionally, to transmit data, it has a transmission unit, which can send and receive data to also act as a relay. An energy harvesting wireless sensor node, as the name suggests, furthermore has an energy harvesting unit (e.g. solar panel), as well as a battery to allow it to store overflowing energy and save it for times, where there is little harvest. This is especially important for methods of harvesting that can have phases, where so little energy is harvested, that not even basic operation of the node can be assured, for example solar energy or wind energy. In order to facilitate decisions based on future energy harvest, the node should also have a prediction unit, that can serve predictions of how much energy will be harvested in a certain time span to the control unit or scheduler, that takes the decisions. These decisions are, when to spend how much energy and when to save energy and the energy is distributed to the consumers by the control unit. Prediction unit, control unit and processing unit can all be seen as software parts of the system that run on the CPU, while sensing, transmission and storage are part of the system hardware. A more in-depth view of the entire system model, including the data and energy flow between the separate parts, can be seen in Figure 3.1.

3.1.1 Energy Neutrality

In order to assure long term system operation, it has to be prevented, that the system runs out of usable energy at any time. The model used to assure this is based on the idea of Kansal et al. [31] to ensure, that the battery charge at the end of a defined time span is at least at the level it was at the beginning of the time span. Also, similarly to their model, the time span is divided up into N different time slots of equal length.

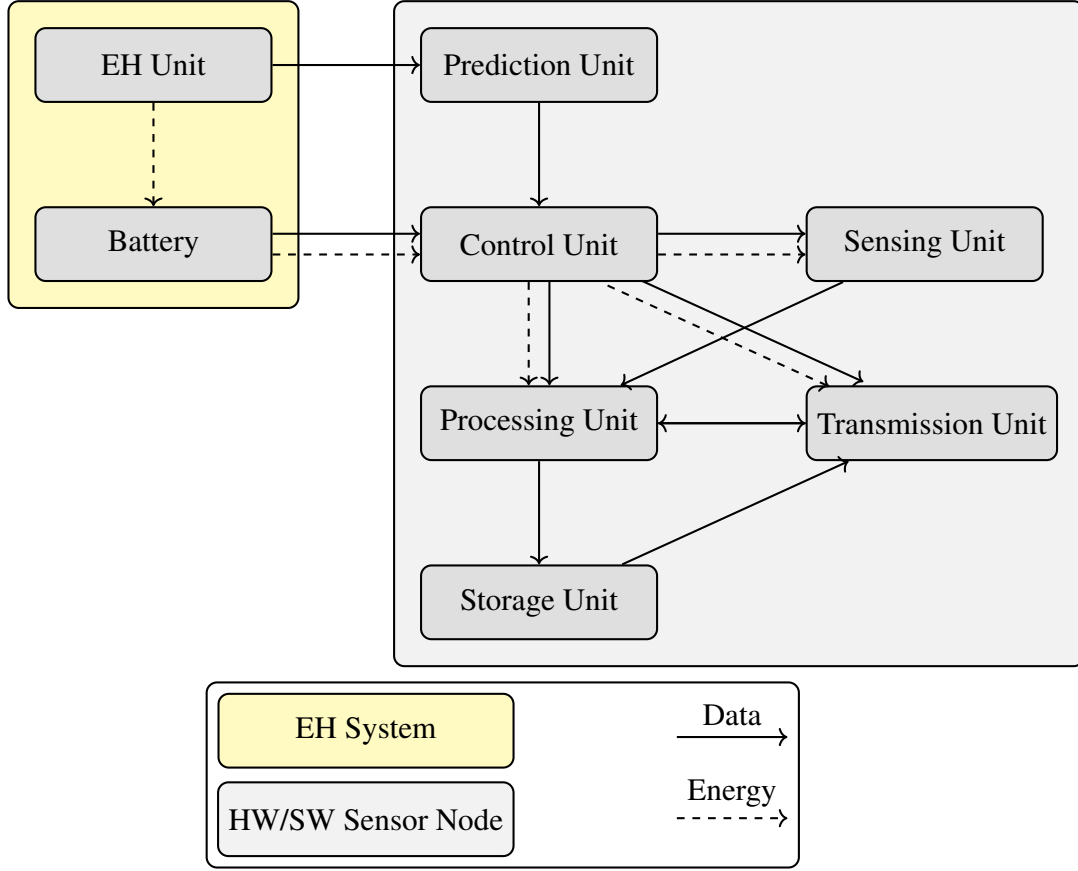


Figure 3.1: System model of an energy-harvesting wireless sensor node.

In one time slot, a specific performance level is assigned by the control unit, that is kept for the entirety of the slot, therefore using a predetermined amount of energy. Additionally, instead of requiring a minimum performance level, a predefined minimum battery level is supposed to never be exceeded after any time slot, to facilitate constant operation of the node. The standard time span chosen in this paper is one day, it can be divided up into 24 time slots of length 1 hour, for example.

3.1.2 Tasks

The unit used to scale node performance in this thesis is referred to using the term "task", using a similar model to Caruso et al. [9]. A system has a predetermined set of tasks $\Theta = \{\tau_1, \dots, \tau_{|\Theta|}\}$. Each one of these tasks τ_i is defined by a unique energy consumption in one time slot e_i and a QoS value q_i describing the node's performance or the utility gained in a time slot when this task is executed. It has to be also noted, that these tasks are modeled independently of any functionality, meaning there are no mandatory tasks or tasks dependent on each other. Moreover, exactly one task is

assigned to a time slot, one specific task may be assigned to multiple time slots or to none at all. The QoS value is supposed to be related to the energy consumption of the tasks. Any task τ_i that is dominated, meaning there exists a task τ_j with $e_i \geq e_j$ and $q_i < q_j$ or $e_i > e_j$ and $q_i \leq q_j$, is declared inefficient and can subsequently be eliminated from the set. Additionally, there is supposed to be no "empty task" without energy consumption, so constant node operation is guaranteed.

This abstract task model allows for multiple interpretations and use cases. In the simulation for example, fixed duty cycles will be used for sensing, resulting in multiple tasks. Their energy consumption is calculated from the energy consumption the sensing function consumes, if it is executed multiple times at a specific rate over the span of one time slot. The QoS value can then either just be the duty cycle, or a function taking the duty cycle as an input, as an example. Different interpretations, including using different hardware parts or algorithms and using Voltage Scaling to reduce the processing rate can also be applied to the model.

3.2 Energy Model

3.2.1 Energy Harvesting System

Any energy harvesting unit and method is generally considered for the algorithms presented in this thesis, with all of them having some common characteristics. Most importantly, no energy harvesting method is perfect, therefore they have an efficiency value η , derivable from the incoming power P_i and a maximum output power P_{max} (both in Watts), by forming the fraction $\frac{P_{max}}{P_i}$. Taking solar energy as an example, P_i would concretely be calculated by the solar irradiance D (W/m²) and the solar cells area S by calculating $P_i = D \times S$. Therefore, the output power supplied by the harvesting unit can be received by computing $P_{out} = \eta \times P_i$. For most simulation results in this paper, a solar energy harvesting sensor node will be chosen. Additionally, one simulation based on wind energy harvesting will also be conducted.

3.2.2 Battery Model

As energy needs to be stored for usage in times of low harvest, which is especially necessary for harvesting methods that have longer times of no incoming energy, some means of storage is needed. For this purpose, batteries and ultracapacitors exist. In this thesis batteries were chosen for the sake of simplicity. A battery is characterized by the elements it is made out of, which also determine some parameters like the nominal voltage V_n and its energy density. For the system, the maximum and minimum charge are also important parameters, the second being determined by its minimum and maximum output voltage V_{min} and V_{max} and the capacity B_{max} . It is determined as follows: $B_{min} = B_{max} \times \frac{V_{min}}{V_{max}}$. When this minimum is exceeded, the node

would have to be shut down. All energy that is harvested by the system while the battery is full is lost, unless it is immediately used. As the charging voltage for the battery has to be higher than the battery's nominal voltage, a DC-DC converter has to be used to increase the voltage, the loss of energy caused by the converter is however neglected. It also has to be noted, that batteries usually have a self discharge rate, which makes them lose the stored energy over time, even if it is not used. In addition, they also have a specific charging efficiency η_c , so not the entirety of the harvested energy is stored in the battery, but some is lost, storing only $P_{stored} = \eta_c \times P_{out}$. These inefficiency parameters, as well as the energy consumption caused by the DC-DC converter are not considered in this model and the simulation, to ensure the model stays as simple as possible. It is also not hard to adapt a system model tested in the simulation without these parameters, as information on them is usually available and they are constant. Consequently, the possibility to bypass the battery in order to directly use the harvested energy, which would be more efficient in practice, is also obsolete. Battery aging, meaning that the maximum capacity decreases with each charge-discharge cycle is also not part of the simulation, as it only takes effect after a fairly high number of cycles. In this model, all charge values, and therefore also the battery charge are displayed in a discrete format, such that all available charge states are integers. The granularity for the discrete "energy slots" can then be chosen based on the desired accuracy. A small granularity would be, for example, 1 energy slot corresponding to 1mAh, but it would also be possible to have 1 slot be 20mAh, so that a battery with a capacity of 1000mAh would have only 50 charge states it could be in. A coarser granularity would also lead to an increasing amount and magnitude of rounding errors, that may need to be accounted for.

The State of Charge (SoC) of the battery depends on how much energy it gets from the energy harvesting unit and how much is used for the node's operation. This relationship can be summarized using the equation $B(T + \Delta t) = [\min\{B_{max}, B(T) - Q_c(T, T + \Delta t) + Q_h(T, T + \Delta t)\}]^+$, $B(T)$ being the charge of the battery at time T and $Q_c(t_1, t_2) / Q_h(t_1, t_2)$ being the charge consumed/harvested in the time span between t_1 and t_2 . The function $[\cdot]^+$ in this case ensures that the charge can never be negative. For the discrete time slot model, only the battery states between the actual slots are actually being looked at, so the equation breaks down to $B_T + 1 = [\min\{B_{max}, B_T - Q_C(T + 1) + Q_H(T + 1)\}]^+$, B_T being the energy after the T th time slot and $Q_C(T + 1) / Q_H(T + 1)$ the charge consumed/harvested during the $T + 1$ th time slot. The incoming charge, as already stated, depends on the energy harvesting unit and the environment. On the left side of Figure 3.2 the average energy harvested in time slots of 1 hour on the 1st of April 2023, for a solar energy harvesting node placed in Greenwich, UK can be seen, the right side in the figure depicts, how it affects the level of charge of a simulated battery, assuming there is no energy being drawn.

To model the power consumption, a simple, but not entirely realistic model is chosen. The discharge rate depends on the summed up currents drawn by all active system

components. For the sake of simplicity, it is assumed that the time a full battery takes to discharge at a constant current I_c is $t_d = \frac{B_{max} - B_{min}}{I_c}$. This model, however, disregards the varying battery voltage during discharge. The battery is assumed to support all charge and discharge currents required by the node, so no current limits are modeled.

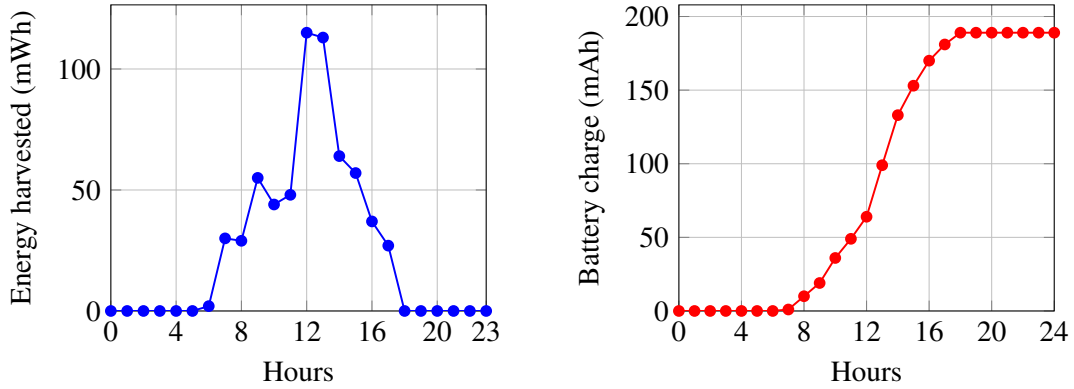


Figure 3.2: Hourly harvested energy (left) and resulting battery charge (right).

3.2.3 Energy Harvesting Prediction

The concrete method to predict the energy harvesting values used is not important, but it should be able to do two specific things. Firstly, it should be able to return the average predicted power harvested in a certain time slot $P_h(t_1, t_2)$ in Watts, from which the average energy harvested in this time slot can be calculated: $E_h(t_1, t_2) = \eta \times P_h(t_1, t_2) \times (t_2 - t_1)$. Secondly, for a specific time slot, it should be able to supply multiple predictions, which usually get more accurate the closer in time the slot is in to the present. The overall goal is to receive a prediction of average power harvested in multiple discrete, equally sized time slots in a specific time span and then during the time span getting more accurate predictions for the remaining slots if needed. In the case of solar energy harvesting, it is also possible for the prediction method to return the average solar irradiance D , it is possible to gain the amount of harvested energy from that as follows: $E(t_1, t_2) = D(t_1, t_2) \times S \times \eta \times (t_2 - t_1)$, with the solar cell area S . When using wind energy harvesting, the wind speed has to be predicted, from which the power output can be computed: $P = \frac{1}{2} \rho A C_p U^3$, $\rho \approx 1.225 \frac{kg}{m^3}$ (at sea level) being the air density, A being the turbine swept area (πr^2), C_p the efficiency and U the wind speed. From this, the energy harvested in a time slot is calculated, by taking the wind speed at the beginning of a slot and assuming it is the average over the entire slot, for example.

3.3 Problem Description

Overall, as has become obvious, the general goal is to find an assignment of tasks for the sensor node in future time slots, such that the system's summed up QoS is as high as possible over the time span and the battery level at the end is at least as high as at the start of the span of time. To formalize this problem description, it will be presented as an Integer Linear Program (ILP) in the following, based on the ILP presented by Caruso et al. [9]. The choice of an ILP over an optimization problem is necessary for the problem to be transferrable to a graph model, therefore discrete battery charge states were also chosen.

$$\max \quad z = \sum_{i=1}^N \sum_{j=1}^{|\Theta|} x_{ij} q_j \quad (3.1)$$

$$\sum_{j=1}^{|\Theta|} x_{ij} = 1 \quad \forall i \in \{1, \dots, N\} \quad (3.2)$$

$$B_{i+1} = \min\{B_{\max}, B_i + E(i+1) - \sum_{j=1}^{|\Theta|} x_{ij} e_j\} \quad \forall i \in \{0, \dots, N-1\} \quad (3.3)$$

$$B_{\min} \leq B_i \quad \forall i \in \{0, \dots, N\} \quad (3.4)$$

$$B_0 \leq B_N \quad (3.5)$$

$$x_{ij} \in \{0, 1\} \quad \forall i \in \{1, \dots, N\}; \forall j \in \{1, \dots, |\Theta|\} \quad (3.6)$$

In the ILP, N denotes the number of time slots and $|\Theta|$ the number of different available tasks. x_{ij} is a binary variable 3.6, that is 1 exactly when task τ_j is assigned to time slot i . 3.2 also determines, that exactly one task is assigned to each slot. A battery value B_i references the charge of the battery after the i -th time slot, B_0 is therefore the initial battery charge and B_N the charge after the last slot in the time span. The vector E contains the energy harvesting predictions for each time slot, starting at $E(1)$. Having clarified this, it is easy to see that 3.1, being the optimization goal, displays the maximization of the summed up QoS values of the tasks assigned to the time slots. The battery in each time slot can be calculated, as shown in 3.3, based on the battery left by the time slot before, the energy harvesting prediction in the slot and the cost of the chosen task, being limited by a set maximum battery charge $B_{\max}$. 3.4 and 3.5 finally show two more constraints, the first stating that the battery charge between time slots must not go below the minimum $B_{\min}$ and the second meaning that the charge after the last time slot has to be at least on the same level as before the first slot.

4 Methodology

In this chapter, the graph model that was developed and used will first be explained. Then, it will be shown how existing graph search algorithms were applied to the problem described in the chapter before.

4.1 Graph Model

For the sake of better understandability, the energy budget management problem will first be explained as a game tree. The problem can be thought of as a game, where the game states are determined by the time slot and the decisions/moves that are taken in each time slot. A task represents a possible move in the game, which would make the system change states, the time slot being incremented and the battery level changing according to the task's energy consumption and the incoming harvested energy, which depends on the time slot. In one state, there may also be multiple possible moves, as it may be possible to assign different tasks to the next time slot. This game can then be depicted in a game tree, where the states are represented as nodes and the moves are the edges connecting the nodes. The tree has a root, the initial system state, middle nodes being connected to exactly one parent and multiple children, and leaves, all possible final states in the last time slot. In the model, the time slot of a state/node is represented as its distance (in nodes) from the root node. This means that for N time slots, the game tree also has a depth of N . The battery level is also saved in the node and is represented as an integer with the chosen discrete granularity. A node v also has a unique quality sum, which can be calculated by adding up the QoS values of the tasks leading from v to the root. Alternatively, the quality sum can also be stored in the node, so it does not have to be calculated when trying to retrieve it.

A simple example such a game tree is shown in Figure 4.1, for a problem instance with $N = 2$ time slots and a set of 2 tasks $\Theta = \{\tau_1, \tau_2\}$ with $e_1 = 1, e_2 = 4$ and $q_1 = 2, q_2 = 4$. Additionally, the vector E with energy harvesting predictions for each slot looks as follows: $E = [3, 2]$. The initial battery level $B_0 = 1$ and $B_{min} = 0$. The Q values annotated at the edges show the quality sum of a particular path and could in this case also be stored in the nodes. To gain a solution assignment from this tree, one simply has to choose a leaf node with battery level ≥ 1 and ensure that the path from it to the root includes no node with battery level < 0 while listing the tasks in the edges on the path to the root for the schedule and reversing the list at the end. A

major drawback of this approach is that it takes very long to create the entire tree and uses a lot of memory for reasonably high N , $|\Theta|$ and battery granularity. As energy predictions vary between different time spans, it might be necessary to rebuild or at least update the entire tree for every time span.

This issue can be somewhat mitigated by transforming the tree structure to a graph. As can be seen in Figure 4.1, there are 3 energy neutral solutions, with 2 of them having the same battery value ($B=1$). In cases, where multiple moves from different states end up with the same battery value in the same time slot, they could, instead of pointing to different nodes with the same battery state, all point to one node. This could massively reduce memory usage, but the quality of a specific path would have to necessarily be calculated by traversing the path back to the root, as one node may be part of different paths with different quality sums when reaching the node. Nodes that have a battery level lower than B_{min} can furthermore be trimmed, as any path/solution continuing from it would be invalid anyway. The revised structure, now a DAG with root ($timeslot = 0$) and leaf nodes ($timeslot = N$) instead of a tree, can be seen in Figure 4.2. The strategy to find an applicable solution from the entire graph also changes, as there can be multiple valid paths to the root from one leaf node. In order to find the best one, all of them would have to be tried in the worst case.

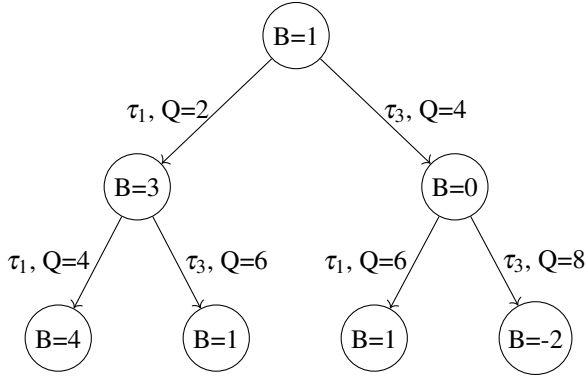


Figure 4.1: Game tree for the energy-budget scheduling problem.

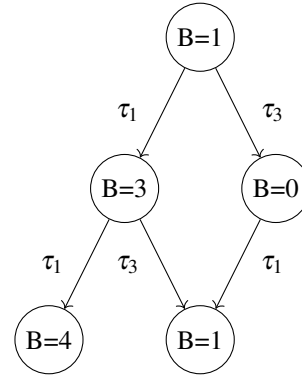


Figure 4.2: Revised game graph with collapsed nodes.

A problem posed for the naive approach of finding a solution, which was described in the above paragraph, is that due to changing energy harvesting predictions for different time periods, the graph would also have to be built from scratch or updated each time. As this alone already takes a lot of time and the full graph is often not even necessary to find a solution, the algorithms presented in the following sections all resort to building the graph and searching a solution at the same time.

4.2 A* Algorithm

The standard version of the A* algorithm aims to find the shortest path in a graph from a target node to a goal node. In case of the energy management problem, the aim is to find the path with the highest quality from the initial state (root node) to a leaf node ($timeslot = N$) with battery $\geq B_0$. Therefore, all leaf nodes satisfying this constraint can be considered the goal node. A*, as explained already in the literature review, maintains multiple paths at once and decides which path to extend based on a sorted priority list. The priority is calculated by taking the highest sum of the quality a path already explored $g(i)$ and the estimated sum of quality from the last node in the explored path to the goal node $h(i)$. This estimation is supplied by a heuristic, that has to be admissible for the algorithm to return an optimal solution. In this case admissibility means that the distance to the goal node is never underestimating the sum of quality when reaching the goal, as the algorithm should search for the highest quality path. An easy way to guarantee this is to take QoS value of the task with the highest QoS and multiplying it by the number of remaining time slots. Also, when a path is extended, it might be the case, that the state to be added to the path already exists as a node. This is why there has to be some way to directly access existing nodes using their time slot and battery level, to check whether a node has to be newly created or not. An example for this can be an array of the size $B_{max} - B_{min} + 1 \times N$. Finally, when adding nodes, the battery constraint B_{min} also has to be respected. Paths that would lead into a state with $B_i < B_{min}$ are not explored further and removed from the priority list. One iteration consists of finding the best open path in the priority queue and removing it from there, marking the last node in the path as visited (so it will not be explored further) and first checking if an admissible solution is found. This is done by checking the time slot of the last node in the path and the battery level. If it is a full, energy neutral path, it is reconstructed and returned. To reconstruct the path, each node has to memorize the parent node the best (partial) path to this node continues at, or alternatively entire paths would have to be saved in the priority list. Else, it iterates over all tasks, that do not put the system into an energy underflow state and checks, whether the state it would lead to already exists as a node. If it does and the new path to the node is better, the node and priority queue are updated. In case no such node exists yet, the node is created and put into the priority list. The algorithm iterates until there are no more open paths, or the goal node was found.

4.3 Topological Sorting

As already stated in the literature review chapter, topological sorting can be applied to the graph model, as it is a DAG. When pairing topological sorting with a primitive graph search algorithm like DFS, it is possible to find the best path to a solution node in $O(|V| + |E|)$, even faster than the A* algorithm. The idea is to perform DFS first

to find a topological ordering and then look for the best path from the source. The graph in itself is also built during the sorting phase. Finding the best path is done by first setting the distance of all nodes to $-\infty$ (except the source) and then traversing the order updating the distance of every adjacent node to the one currently processed. A pseudocode representation for this can be found in Algorithm 1. To find the best solution, the QoS value of the tasks in the edges have to be respected, while also requiring that the goal node has a battery equal or higher than B_0 . At the end, the leaf node fulfilling this requirement, which has the highest *quality* sum value is chosen. To find the task assignments, the path back to the root, guided by the *best_parent* reference has to be followed. To avoid confusion it should be made clear that the code refers to the quality values of the singular tasks as q and to the summed up QoS values of a path as *quality*.

Algorithm 1: Get best leaf node from topological order

Input: *top_order*, N , B_0
Output: Best energy-neutral leaf node

```

1 leaves  $\leftarrow \emptyset$ ;
2 foreach node  $u \in \textit{top\_order}$  do
3   if  $u.\textit{timeslot} = N$  and  $u.\textit{battery} \geq B_0$  then
4     Add  $u$  to leaves;
5     continue;
6   foreach edge  $e \in u.\textit{outgoing}$  do
7      $v \leftarrow e.\textit{child}$ ;
8     if  $v.\textit{quality} < u.\textit{quality} + e.\textit{task}.q$  then
9        $v.\textit{quality} \leftarrow u.\textit{quality} + e.\textit{task}.q$ ;
10       $v.\textit{best\_parent} \leftarrow u$ ;
11 return  $\arg \max_{l \in \textit{leaves}} l.\textit{quality}$ ;
```

It is clear to see from this algorithm, that this step only has a time complexity of $O(|V| + |E|)$. As DFS, or any other method to get the topological order of a DAG also takes time $O(|V| + |E|)$ and reconstructing the path only takes $O(N)$, the solver overall also has a time complexity of $O(|V| + |E|)$. While the A* and the topological sorting based algorithm yield an optimal solution, they are usually even slower than the Dynamic Programming solver from [9], as they additionally need to build a graph structure. Therefore, approximate algorithms will now be investigated.

4.4 Beam Search

For adaptation of the Beam Search algorithm, multiple changes had to be made to the standard variant. Beam Search, as already explained, works similarly to BFS,

but only searches a part of the adjacent vertices of each node, instead of all of them. How many neighbors of each node are searched is determined by the *beam_width* parameter, which has to be equal or lower than the amount of available tasks to be in accordance with the graph model. The algorithm iterates through the layers of the graph, which in this case are the time slots. When it reaches the end, all leaf nodes that were found by the algorithm are returned, the best one with battery $\geq B_0$ is selected and the path is reconstructed, similar to how it works in the topological sorting based algorithm. To be able to select the best leaf node, each node needs to store the quality sum of the best path leading to it and a reference to the parent node that is part of the path. Neighbors are selected in regards to the quality of the task in the edge that connects them and the level of the battery that would be left in the next time slot, if this task was selected. A general example for the Beam Search algorithm can be found in Algorithm 2. **Expand** adds all edges to adjacent nodes and creates them, if they do not exist already, updating their *quality* of the best path found and *best_parent* if applicable. **Evaluate** does the selection of best neighbors as explained before.

Algorithm 2: Beam Search for Energy-Aware Task Scheduling

Input: $root, N, B_0, B_{min}, B_{max}, \Theta, beam_width$
Output: Best energy-neutral leaf node

```

1 Initialize  $nodes[N][B_{max} - B_{min} + 1] \leftarrow \text{null}$ ;
2  $current\_layer \leftarrow \{root\}$ ;
  // Assert  $beam\_width \leq |\Theta|$ 
3 for  $t \leftarrow 0$  to  $N - 1$  do
4    $next\_layer \leftarrow \emptyset$ ;
5   foreach node  $u \in current\_layer$  do
6     Expand( $u, N, B_0, B_{min}, B_{max}, \Theta, nodes$ );
7      $best\_children \leftarrow \text{Evaluate}(u, N, B_0, B_{min}, B_{max}, \Theta, nodes)$ ;
8     foreach child  $v \in best\_children$  do
9       if  $v \notin next\_layer$  then
10        Add  $v$  to  $next\_layer$ ;
11    $current\_layer \leftarrow next\_layer$ ;
12 return ( $current\_layer, root$ );
```

A concrete implementation of the **Evaluate** function may be started by checking if the selected task τ_i in the edge leads to a state, which has an equal or higher level of battery than B_0 . If it has, it takes the raw q_i value of the task τ_i as evaluation parameter for this neighbor. Otherwise, returns a penalized version of q_i , for which it makes sense to select an exponential function, so moves leading to lower battery states are averted if possible. Concretely, the initial implementation for this penalty

function was first conducted as follows, with B being the battery state of the proposed node:

$$penalize(B) = quality - quality \times (1 - e^{(-k) \times \frac{B_0 - B}{10}}) \quad (4.1)$$

The further B is away from B_0 in this function, the closer the e^x function will come to 0 and therefore it will become closer to calculating $quality - quality \times 1 = 0$. The parameter k can additionally be adjusted to scale the impact of solutions that are far of the battery start value. If chosen higher, they are penalized more strictly, if k is lower, they are not treated as harshly. A good middle ground was found at $k = 10$.

Overall, this approach can be seen as a greedy algorithm, that weighs off quality and battery state in every decision it takes. However, this property can lead to a massive drawback, namely, that the algorithm is often not saving enough energy in phases of high harvest for it to allow operation throughout low harvest phases. This is due to the decision for a task to be chosen only being taken in regard to the next state and the next harvesting prediction, completely disregarding any future states. As a consequence, the algorithm might sometimes fail to find a solution, as there is not enough energy left at the beginning of the low harvest phase to sustain continuous node operation without causing the battery to drop below B_{min} , even when choosing the cheapest task for all time slots. One possible approach to this problem might be the implementation of a "look-ahead" functionality, that allows the evaluation to also consider the energy harvesting predictions of the next $n \leq N$ time slots and forcing the algorithm to save energy if the predicted values drop in the future.

On the other hand, it might also be possible to adapt the threshold for penalization to the harvesting prediction's profile, which means that the condition whether a penalty is applied is no longer simply checking, if the proposed battery level is below B_0 , but below a threshold, that is individually determined for each time slot. This threshold does not completely influence the amount of battery a solution should have in a certain time slot, but merely gives a guideline. Moreover, the threshold obviously should not drop below B_0 , so the algorithm is at least guided towards an energy neutral solution. In the last time slot it should always be set to B_0 however, to allow the algorithm to spend any excess energy. The decision, which option to implement was made in favor of the second variant, as it would impact the execution time much less. The procedure to calculate the adapted thresholds is started by identifying all regions of low harvest in the prediction profile and saving their start and end slot. The threshold is now linearly increased in the slots after the first slot (or the first end slot of low harvest if it is before the first starting slot), leading up to the starting slot of the next low harvest phase. After that, it is linearly decreased again until the end slot of the low harvest region, or the last time slot, reaching B_0 again. The peak value for the threshold has to be chosen in regards to the battery level and how much energy can be harvested. In some cases it might make sense to set the peak to B_{max} , as an example. This small addition causes the algorithm to become much more stable.

4.5 Monte-Carlo Tree Search based Algorithm

The basic concept of MCTS was already explained in the Literature Review section, but to recapitulate, it goes over a defined amount of iterations, which are set by the programmer or user to build a tree, or graph in this case. In each iteration, the algorithm goes through 4 phases and, in its base variant, adds exactly one node and one edge to the tree. The phases can be summarized as (1) going down the most promising existing path (selection), (2) adding one node when reaching a node that can add another child (expansion), (3) simulating a full, random playout and reaching a value describing the success of the playout (simulation) and (4) backpropagating that value along the path chosen down the tree up to the root (backpropagation). Every iteration is started from the root node, which, in case of the presented graph model, is the node with $timeslot = 0$ and $battery = B_0$.

In order to adapt to the graph model, the selection phase was changed to only stop, when a completely new node is added to the graph, meaning, that when it detects that another child can be added in the current node, it first checks if this state already exists as a node (e.g. in an array with node references). If it does, only the edge to the child is added and selection is continued. In the simulation phase, the "simulation value" reached at the end is the summed up quality of the path taken in selection, expansion and simulation together. Additionally, it has to be tracked, whether the solution presented by the path is energy neutral. If it is, the quality sum value can be returned without changing it. If it is not, however, the algorithm could return 0, but this would not make much sense in case of a very good solution, that is almost energy neutral, as it still contains some valuable information for the remaining iterations. Therefore the concept of penalizing solutions that are not energy neutral was introduced. The penalty is calculated similarly to the penalty function presented in the Beam Search section, this time taking the summed up quality value of a path as input. For this case, the algorithm was found to work best with parameter $k = 1$. In case the simulation reaches a state, in which the battery is below B_{min} , 0 is returned. In the backpropagation phase, the path up to the root is no longer obvious, as it would be in a tree. From the possibilities for backpropagation presented in [61], the one only targeting the selection and expansion path was chosen, as the algorithm should return one specific path as solution and because it is faster. Consequently, the path taken in selection and expansion has to be memorized. As building a full path is the goal, the algorithm always saves the best explored full path from one iteration, that was traversed in selection, expansion and simulation phase and added together. The best path is characterized as leading to a simulation end state or leaf node with battery level $\geq B_0$, that has the highest summed up quality value of the tasks assigned. In case no energy neutral path was found during any iteration, no solution is returned. In addition to these modifications, there were some less obvious changes that were made to improve the efficiency of finding a solution for the path finding problem. The formula used for the selection of the best child node in the first phase was changed,

completely removing the exploration parameter, causing it to be just $\frac{w_i}{n_i}$ (average simulation result) and making it find a solution much faster. For the same reason, the expansion phase was changed to add all missing child nodes to the last node found in selection and then initiating one simulation per added node. The idea for this was based on the simulation in parallel MCTS [11], increasing the efficiency of one iteration and causing more paths to be explored. In the backpropagation phase, the sum of all simulation results is then applied to the descent path from selection. Finally, as the tasks chosen in the simulation phase are random, energy overflows, where the battery would reach B_{max} and incoming energy is therefore wasted, might occur. To mitigate this, the simulation in this case was manipulated to choose a task with a higher cost. An overview over the revised MCTS phases can be seen in Figure 4.3.

Another improvement that may seem sensible to improve the algorithm's efficiency is to look at the part of the end solution, that was randomly generated in the simulation and, if there is more energy than B_0 in the battery at the last time slot, to upgrade some randomly assigned tasks to spend it. This would only be initiated once, after the last iteration and therefore not take too much time. While it may improve the solution of one execution of the algorithm, in a real-life scenario it usually does not really make a difference, as the system can also spend the excess energy in the next time span.

4.6 Recalculation

All of the presented solvers rely on energy harvesting predictions, which can be inaccurate. To account for this, if too high of a difference between planned battery level and actual battery level slot is detected at a time (can be higher or lower), a recalculation of assigned tasks for the remaining time slots can be initiated. The decision for this has to be taken early enough, to avoid having to shut the node off if the predictions were too high, as this is a scenario considered for neither algorithm. It could theoretically be modeled with a task τ_0 with $e_0 = 0, q_0 = 0$, but is to be avoided, as stated in the Fundamentals section. To start a recalculation, the time slot parameter in the algorithm has to be set to the amount of remaining slots until the end of the time span. The predicted harvesting values for the rest of the slots can also be modified, if a new prediction is available, note however, that the first value in the prediction vector $E(1)$ now corresponds to the next time slot, not the first time slot in the time span. Every algorithm can then be started with a root node, that represents the current battery level and time slot 0. Overall the algorithms can then calculate a reassignment of tasks for only the remaining slots, that aims to hit the goal battery level and avoid exceeding the lower limit B_0 .

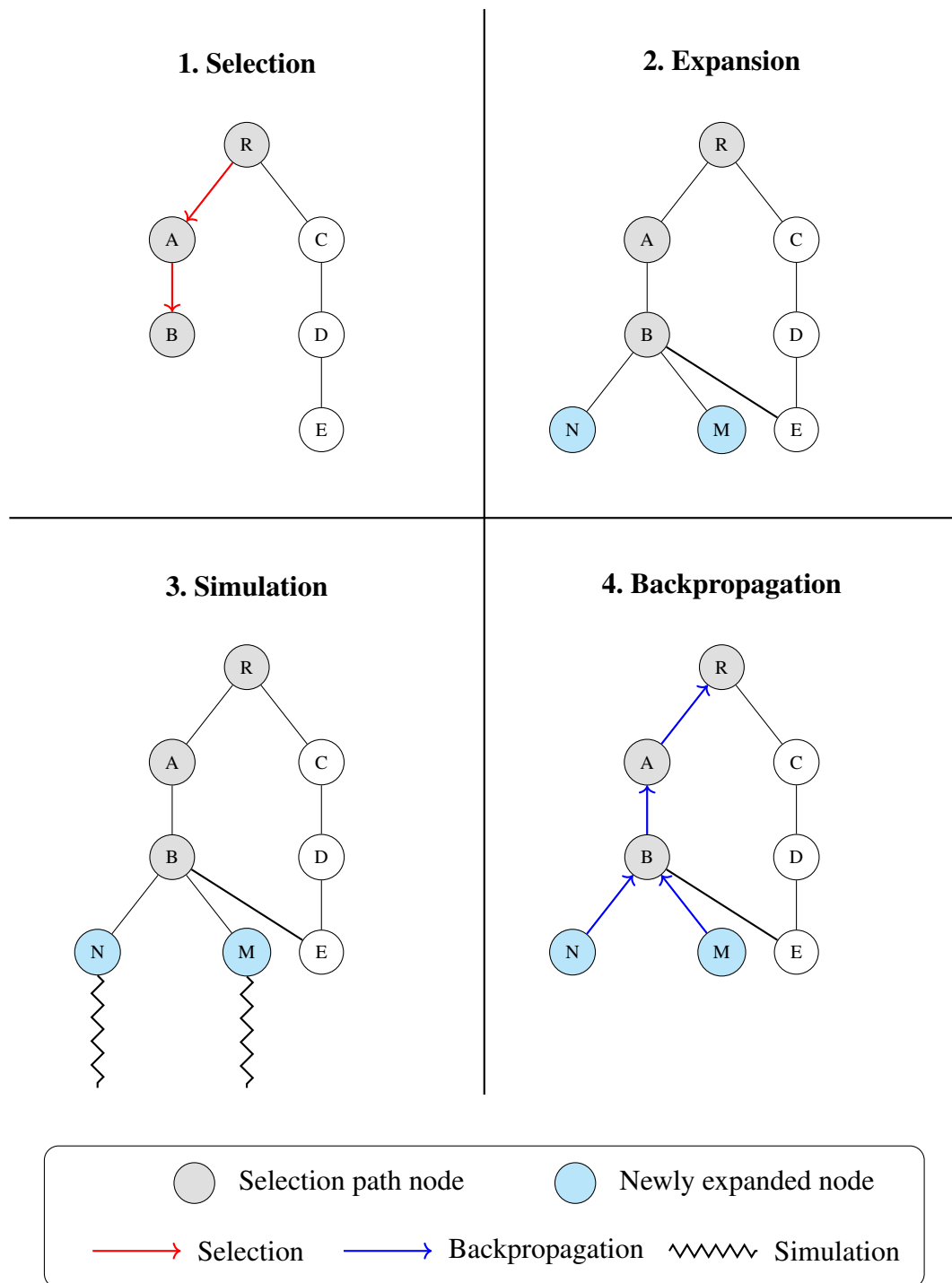


Figure 4.3: Phases of problem-specific MCTS adaptation



5 Results

5.1 Simulation System Parameters

In most of the simulations, solar energy harvesting will be used as the chosen method. For this purpose, a solar energy harvesting system with the parameters displayed in Table 5.1 is simulated in Greenwich, UK (51.477, 0). Additionally, one simulation will be conducted with values from wind energy harvesting, for which a module with rotor diameter of 0.5m, efficiency of 50% and a rated wind speed of $12 \frac{m}{s}$ is used to calculate the output energy, also simulated in Greenwich. The concrete rechargeable battery chosen for all simulations is a Keppower 14500 - 1000mAh lithium-ion AA battery. As its name suggests, it has a maximum capacity of 1000mAh and, with its nominal voltage V_n being 3.7V, can store $1Ah \times 3.7V = 3.7Wh$ of energy. Its output power is a maximum of 4.2V and 2.5V at a minimum. Consequently, the minimum energy that has to be maintained, so the node does not have to shut down is $3.7Wh \times \frac{2.5V}{4.2V} \approx 2.2Wh$ which is equal to a charge of $\frac{2.2Wh}{3.7V} = 0.595Ah$, rounded to a minimum charge of $B_{min} = 600mAh$ for all simulations.

For the energy harvesting predictions, predicted clear sky irradiance data supplied by the pvlib python library [4], which was configured to use the Isotropic Sky Diffuse Model. This model is very simplistic, therefore for the quality-wise more important tests were conducted using historical irradiance data gathered from pvgis [18], a database containing irradiance data for all of Europe, which includes more variance due to weather. More information on the dataset can be found in [27]. For the concrete values for wind energy harvesting, historic wind speeds from Open-Meteo were used [75].

5.2 Optimal Algorithm Parameters

As already explained in the sections about the specific algorithms, both Beam Search and MCTS have a unique parameter, that has to be set before execution of the program. Beam Search namely has a defined Beam Width, which is equal to or smaller than the number of tasks in the model, MCTS has a specified number of iterations. In order to find a good value for these, a simulation setup with the maximum battery charge of the battery specified in Fundamentals split up into 100 slots was chosen, meaning $B_{max} = 100$, $B_{min} = 60$ and one slot represents the battery charge rounded to

Parameter	Value
Specified Load Voltage V_{ld}	3.3V
Maximum Power Output P_{max}	500mA
Surface Area	36cm ²
Module Efficiency	11%
Surface Tilt	30°
Surface Azimuth	South

Table 5.1: Specifications of the solar cell used for the simulations

the next 10mAh. Additionally, the 1st of April 2026 with predicted energy harvesting values according to pvlib was taken for the energy predictions and the simulation was started at 16:00 on the specified day. The starting battery charge B_0 was set to 80 accounting for a daytime of charging just before starting the simulation. The time span to simulate was set to 1 day, split up into 24 slots of 1 hour each, which is the standard configuration also used in most of the other simulations, unless otherwise stated. The tasks are displayed in 5.2 and were selected randomly, but based on the harvesting values. Results for the simulations can be seen in Figure 5.1 on the left (MCTS) and the right side (Beam Search) respectively. Both diagrams show the resulting quality while varying the parameter belonging to the algorithm. The quality values displayed in the diagram for the MCTS algorithm are averages over 100 executions, as it does not give fixed results due to its random-sampling based nature. The red line at $quality = 100$ in both diagrams represents the maximum sum of quality achievable for the simulation setup, which can be gained from either the A* or the topological sorting based algorithms.

It is clearly visible, that the quality returned by the MCTS algorithm quickly rises until about 10 iterations. This is due to the fact, that before that amount of iterations, it rather often finds no solution and therefore returns 0 as quality, dropping the average. After reaching that value however, the quality plateaus and never actually reaches the maximum achievable quality, which is a result of removing the exploration part of the UCT formula. An example of how the graph would look when including the exploration part of the formula can be found in 5.2. In this case, more possible solutions might be accounted for in one MCTS execution, which may lead to it converging toward the optimal; however, the algorithm becomes significantly more unreliable and takes about 10 times as many iterations to consistently find a solution. This is also the main reason for the design choice to neglect exploration. As a result of the quality-wise performance of MCTS, in the following simulations, the iteration count $N/2$ will usually be used, as it consistently finds a solution and even slightly

Task ID	Energy Cost	Quality
1	4	6
2	3	5
3	5	7
4	8	10
5	2	3
6	1	1

Table 5.2: Task set with associated energy cost and quality value

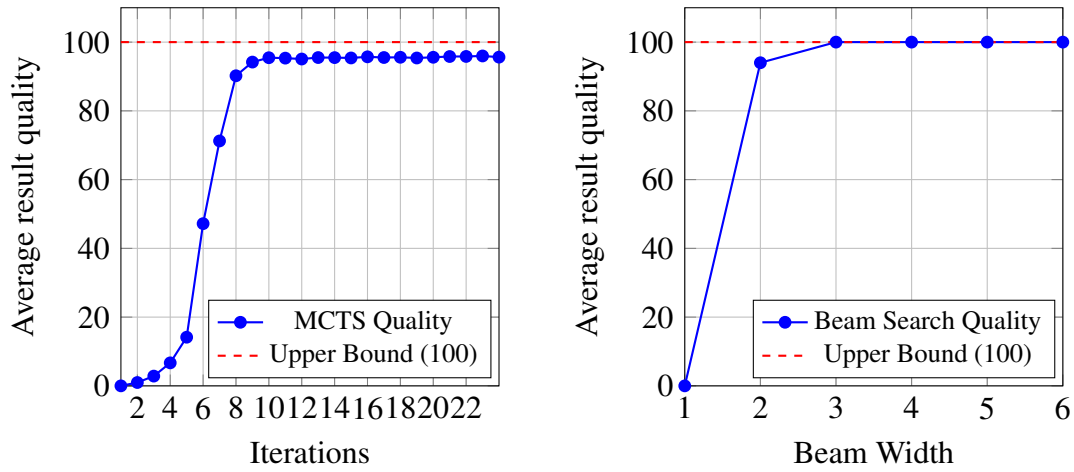


Figure 5.1: Average solution quality for varying iterations/beam width with maximum achievable quality as reference

lies above what is necessary according to the plot. Also note, that if no solution was found after the set amount of iterations, more iterations can theoretically be executed on the existing graph until a solution is found.

The Beam Search algorithm, other than MCTS, always finds the same solution for one specific setting. Therefore the plot on the right side only displays the results of one execution with varying beam width. The algorithm already finds an admissible solution with $quality = 94$ for a beam width of 2, everything above finds the optimal solution. As a result, a beam width of $\lfloor |\Theta|/2 \rfloor$ ($|\Theta|$ being the number of tasks) will be chosen for all other simulations, unless otherwise stated. In contrast to the iterations in MCTS, the beam width cannot be modified after execution, if no solution is found. In this case another full execution with higher beam width value must be initiated. Overall, it can also be seen, that Beam Search can find an optimal solution much more

easily and reliably than MCTS. MCTS of course can also find a perfect solution, it is not visible here, because every data point is an average of 100 executions, but it is also limited by the randomly selected part of the path.

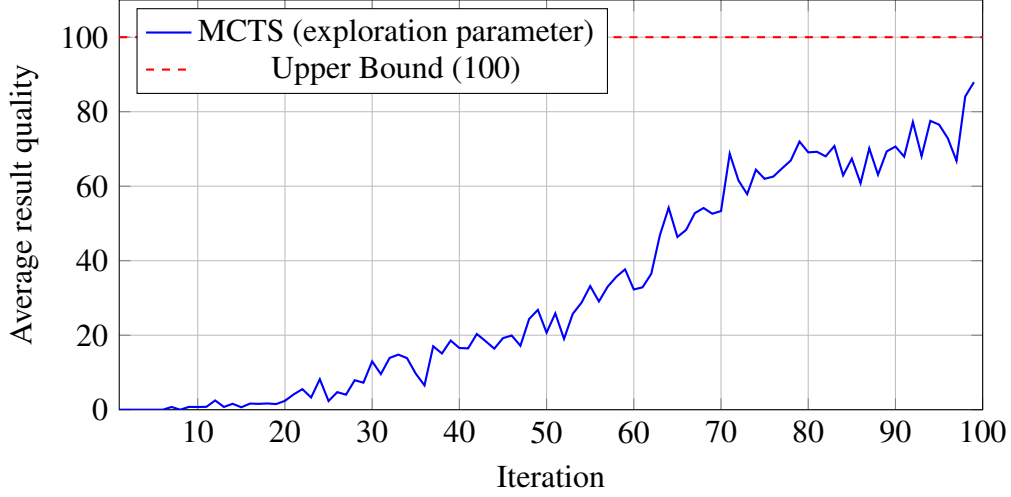


Figure 5.2: MCTS performance over 100 iterations with exploration parameter

Adding to that, the result quality of MCTS is also dependent on when a low energy phase (e.g. night) occurs in the predicted energy pattern. As the simulations randomly assign tasks, they often "get stuck" at these energy droughts, meaning that the simulated battery level goes below B_{min} and 0 is returned as simulation value. Therefore, in some cases, when selecting, all children of a node have a node value of 0, in which case the algorithm sensibly chooses the least expensive task. This usually only happens, until the built graph is expanded far enough into the low energy phase, so the simulations do not cause energy underflows as often. But it also means that the point, from which more solution are found depends on the location of the energy drought in the prediction pattern, because at least part of the end solution is usually created by the simulation. For that reason, the likelihood of finding a good solution might significantly change, when the start time of the algorithm is changed, as the cheapest task is assigned more often. An example for this is provided in Figure 5.3, displaying the plot of iterations and resulting quality for start times 0:00h and 8:00h in addition to start time 16:00h on the left side of 5.1. The same simulation setup was used but B_0 was set to 75 (some energy already used at night) for the start at 0:00h and 65 for 8:00h (low energy after night). In both cases, solutions were found as quickly as when starting at 16:00, but with a higher average difference from the optimal (116 in this case). This is due to the fact, that the drought phase (night) is at the end of the time slots, therefore most simulations before the graph reaches it end in an energy underflow and no path is found. This causes the algorithm to the cheapest task, which explains the increasing quality offset to the optimal solution.

As a consequence, all further simulations using solar energy harvesting predictions will be initiated at 16:00h. Moreover, this could be understood as an indication that MCTS may be more suitable for energy harvesting methods that supply a more constant stream of energy than the sun. Beam Search, on the other hand, does not show such varying behavior.

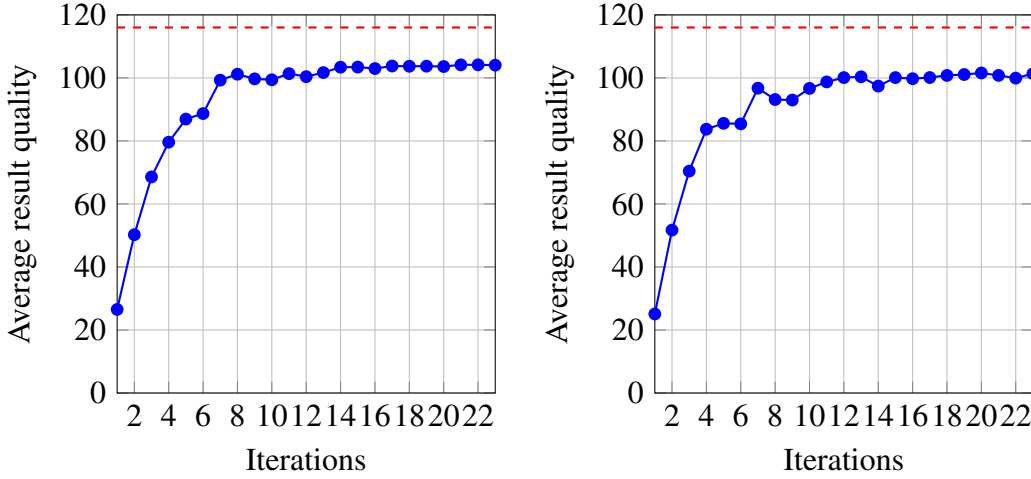


Figure 5.3: Comparison of MCTS performance when starting at 0:00 (left) and 8:00 (right)

Finally, it will be demonstrated that using a function that improves the randomly generated part of a solution assignment returned by MCTS, in order to use remaining energy more efficiently can be employed to improve a single solution, but is generally not necessary when the algorithm is executed for multiple time spans (days) in succession. For this purpose Figure ?? shows over a span of 3 days, with predicted energy harvesting values from pvlib on the 1st to 3rd of April 2026, how the MCTS solver behaves when using or not using the upgrade functionality. The other general setup was the same as before, so $N = 24$ time slots and one battery slot represents 10mAh. It shows the average quality/battery difference of basic MCTS and MCTS with upgrade functionality in the 72 time slots over 100 executions. While the version using the upgrade function ensured a higher operation quality for the first day (negative quality difference after 24 hours), it got overtaken after the second day. This is a result of the other version using the leftover energy at the end of one time span in the next span to gain a higher quality solution for it. At the end, the version without the extra step has accumulated a higher quality and has a higher remaining battery charge. Having understood and evaluated this, the consequence is to disregard the idea of an upgrade function for MCTS entirely, which will be followed for the remaining experiments.

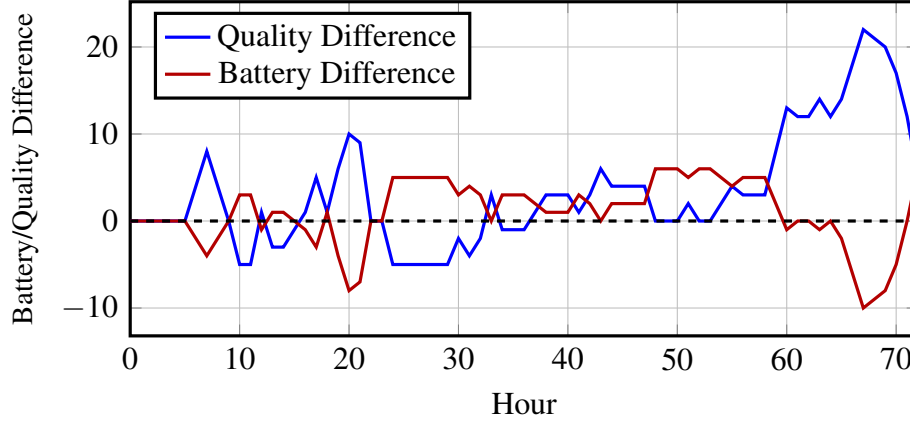


Figure 5.4: Difference in quality and battery for execution of MCTS with and without upgrade functionality (No Upgrade - With Upgrade)

5.3 Performance

After finding out the ideal settings to run MCTS and Beam Search, their performance will now be evaluated. For this purpose, the two algorithms will be compared in regards to their execution time, memory usage, energy usage and quality of result. The reason to not include Topological Sorting and A* **in part** of this comparison is that they produce an optimal solution and are slower than other optimal solvers. In order to have a point of reference for the performance, the DP algorithm developed by Caruso et al. [9] and the Upgrade/Downgrade (UD) algorithm introduced by Escolar et al. [15] will also be part of the comparison. The former returns optimal solutions, while UD also computes an approximate solution. The first comparison that will be presented is how the algorithms' execution time scales when increasing the amount of time slots the day is divided into, displayed in Figure 5.5a. Other than this, the setup stays the same as in the simulations before, the cost of the tasks is scaled $\frac{N}{25} \times e_i$, where e_i is the cost of the task according to 5.2 to account for the changing amount of energy gathered with modified time slot length. The energy prediction is the same as in the other simulation, to adapt it to the changing number of slots the values were interpolated and rounded. All timing experiments were conducted on a Seed Studio XIAO nRF52840, using the `micros();` function to record start and end time of the execution. Only necessary setup steps and the algorithm execution were recorded. Note that the e^x function used in the penalty function of MCTS and Beam Search had to be replaced by a lookup table with linear interpolation, as the `exp()` function was very slow on the chosen board. This did not significantly change the result quality. The two graph based algorithms that guarantee a perfect solution are even slower than DP, Topological Sorting **just barely and** also scaling linearly and A* being much slower and scaling worse. On the other hand MCTS and Beam Search are faster than DP. Beam Search seems to be the faster of the two introduced heuristics, only being

slower than MCTS for $N = 15$. However, they both seem slower than UD in this experiment, with Beam Search only being faster for one specific setup, although overall the difference to UD does not seem too significant.

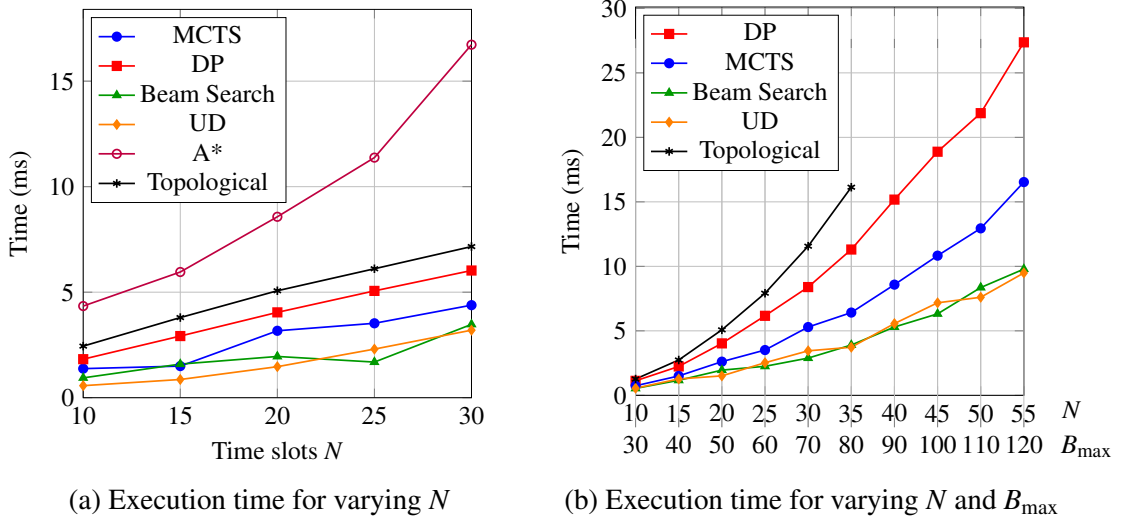


Figure 5.5: Execution time comparisons for selected algorithms

The amount of time slots N is not the only thing affecting the execution time of the algorithms, it can also be influenced by a varying amount of battery slots. For this purpose, the time slot and battery slot count were increased simultaneously. The task costs can remain the same this time, as the energy/battery slots are smaller, thereby also scaling the actual task cost without changing the value of e_i . The starting battery for this experiment was kept constantly at $B_0 = \frac{B_{\max} + B_{\min}}{2}$. In the results in Figure 5.5b, it is again visible that both approximate graph based algorithms perform much better time wise than the DP algorithm, with Beam Search also scaling much better. UD and Beam Search challenge each other, while MCTS lies slightly above them. Overall, both algorithms are an upgrade on DP in this regard, UD on the other hand still provides a challenge. Topological Sorting was also tried in this setting, being slower than DP for the first few values and for more than 35 time and 80 battery slots needed so much memory that it could not be executed anymore on the nRF52840 board.

On sensor nodes, memory is also a limited resource, this is why the memory use of all algorithms will also be **investigated**, using the same scaling parameters as before. For the sake of simplicity, only memory used during the entire execution of the algorithm is considered for this. In case of DP this means the memory used by the two arrays the ILP is solved in, which memorize the best task and best achievable quality for each time and battery slot. In MCTS and Beam Search, the amount of nodes and edges in the graphs is counted and then multiplied by the memory one node/edge uses, which

is specific for every algorithm. The size for these structures varies across the different heuristics, as they need to save different values, MCTS for example saves a sum of all simulation values that were connected to a node and the amount of time a node was visited. Beam Search needs to save a reference to the best parent of a given node and the quality value of the best solution. Additionally, both need to save references to nodes in arrays to be able to find them again while building the graph, which requires additional memory. In contrast to that, UD does not have any real data structure or array to solve the problem in, only a single array of length N for the assigned tasks. Its memory requirement is much lower than the one of other compared approaches as a result, as can be seen in 5.6a. The memory usage of Topological Sorting scales so badly that not enough space was available on the board after 35 time slots, so the data recording was stopped. Overall, due to the graph structure, both approximate graph based solvers have a much higher and much worse scaling memory usage than DP and especially UD, between them is not much of a difference with Beam Search, again, performing slightly better. They are however a massive improvement on algorithms that explore the entire graph in terms of memory use.

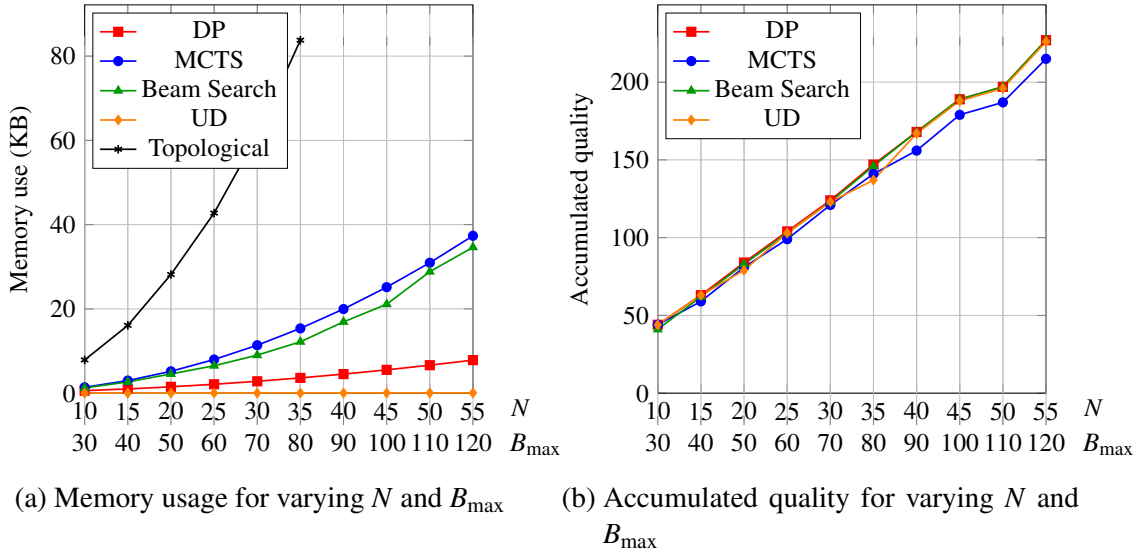


Figure 5.6: Memory usage and result quality comparison for selected algorithms

The result quality was the final metric tracked in this set of simulations, it can be viewed in 5.6b. Beam Search got the closest overall to the optimal solutions computed by DP, being slightly better than UD. MCTS always laid a bit below the other 3 algorithms, with the exception of the first data point. A big downside to UD that can be mentioned here, which occurred during the simulation, was, that it additionally needs a parameter denoting the time slot of "sunrise" and "sunset". These have to be exact and cannot be as easily found as the two points in time, that are used to calculate the threshold in Beam Search, as they have to be a bit before/after the sun

completely disappears/raises. Choosing the wrong parameters, even if only one time slot off, can cause the algorithm to find a significantly worse solution, than would be possible, up to 30-40% lower than optimal. As a consequence, when this algorithm is used over the span of an entire year, with unpredictable weather patterns and actual harvesting values differing from the predictions, this could lead to complications regarding the nodes operating quality and even reliability. A closer look into how the results gained from the algorithm behave when scaling N and the amount of battery slots independently and at the same time **can be gotten** in Figure 5.7. It displays the relative distance of the solution quality received from every algorithm to the optimal one computed by DP, with increasing N and/or the battery slot amount in (a), (c), (e). Additionally, the relative distance to being exactly energy neutral ($B_N = B_0$) is shown in (b), (d), (f). The predictions for 1st of April 2026 were again taken from pvlib. For MCTS, due to its non deterministic nature, the average over 100 executions was considered again. Beam Search was executed with *beam_width* = 2, as it found the optimal solution every time for higher *beam_width*. UD did not show the same good quality results as in the previous tests, having the highest distance from the optimal at a high amount of battery slots, with this being the best possible solution found while evaluating all possible parameter setups for it (sunrise/sunset). Beam Search and MCTS on the other hand stay rather consistent with changing execution variables, with MCTS sometimes leaving some additional energy at the end and Beam Search always using the entirety. The fact that the distance of result quality to the optimal and end battery to B_0 only changed with varying amounts of time slots, not battery slots, in case of Beam Search and UD is also noteworthy.

In order to demonstrate the advantage of especially MCTS when using other means of harvesting energy, a simulation for a wind energy harvesting pattern with 24 time slots was performed, based on historic wind data from the 15th of June 2023. All energy harvesting module parameters were explained in the first section of this chapter, Beam Search was modified to not use its "threshold" logic, as there were no longer phases in the pattern where there was low harvest. UD was specifically only designed with solar energy harvesting in mind, making its use cases somewhat limited. It was tried twice, one time with the specified time slots being adapted to the harvesting pattern and one time to the actual time of sunset and sunrise, but both tries yielded the same result, which is lower than all other methods. Beam Search might have suffered from being restricted from spending more energy before the very high energy phase, and therefore not using the entire available energy as energy was lost. MCTS, on the other hand, found a very good solution on average and used almost the entire budget every time. The comparison of how much lower the quality sum calculated by every algorithm was than the optimal solution is shown in Table 5.3 by displaying the relative distance to the optimal. Alongside the table also shows how far away the battery value at the last time slot was from B_0 , a positive number means it was higher.

Due to the fact that the execution of these assignment algorithms also uses energy

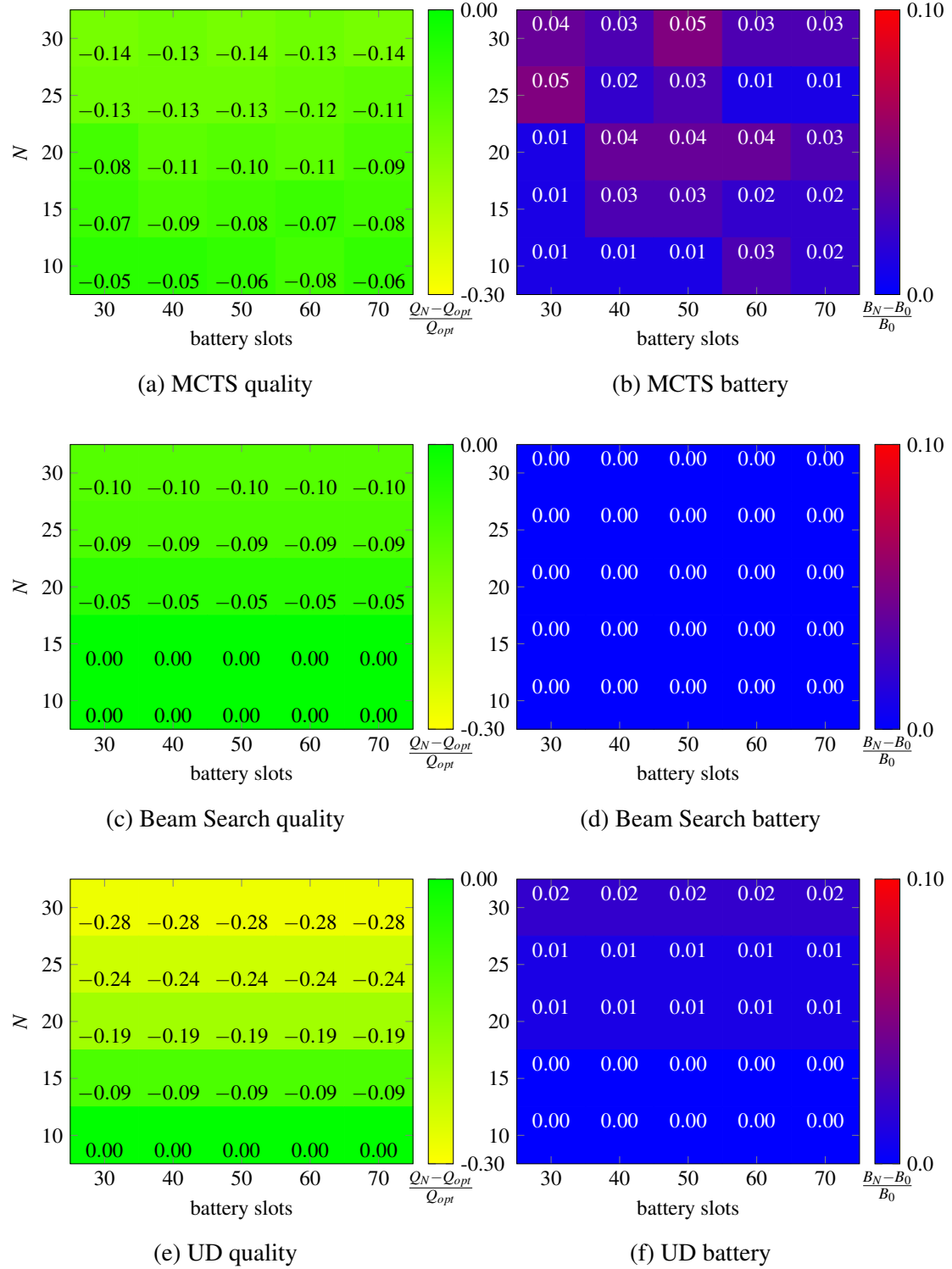


Figure 5.7: Impact of scaling parameters on result quality and battery

Algorithm	RD from optimal	RD of B_N from B_0
MCTS	-2.67%	7.5%
BS	-8.02%	11.25%
UD	-11.22%	20%

Table 5.3: Relative distance from optimal accumulated quality and exact energy neutrality for wind energy harvesting values

and sensor nodes are a very energy constrained system, the final metric that was evaluated in this section is the energy consumption of the four compared algorithms along with any necessary initialization steps. The Nordic Power Profiling Kit II was used to measure the energy consumption of the nRF5284 during the execution of the solvers. It was used in source mode, meaning that it was acting as the energy source for the nRF52840 board, which resulted in more accurate measurements. As parameters for the algorithms, the standard performance evaluation values were chosen, so $N = 24$, $B_{min} = 60$, $B_{max} = 100$ and $B_0 = 80$. Again, energy harvesting predictions for April 1st 2026, provided by pvlib, were used. The results can be seen in Table 5.4, it can generally be stated, that the energy consumption of each algorithm mostly depended on the execution time of the algorithm and the energy drawn during execution was small enough for all algorithms to be negligible in most systems.

Value	DP	UD	MCTS	Beam Search
Average current draw	7.53 mA	6.89 mA	7.24 mA	6.95 mA
Maximum current draw	8.59 mA	8.42 mA	8.69 mA	8.24 mA
Execution time	5.20 ms	2.02 ms	3.89 ms	1.86 ms
Charge drawn	10.87nAh	3.87nAh	7.71nAh	3.59nAh

Table 5.4: Results of measuring energy consumption of algorithm execution

5.4 Recalculation Results

As already stated in the Methodology Section, it is possible for all algorithms to do a solution recalculation in the middle of a time span, in case the actually harvested energy values are lower or higher than predicted. The goal is then either to find a new assignment of tasks to the remaining time slots, that is energy neutral again or uses overflowing energy better. In order to do this in the graph based algorithms,

Task ID	Duty Cycle	Energy Cost	Quality
1	1%	2	2
2	5%	6	4
3	10%	11	6
4	25%	27	10
5	50%	53	14

Table 5.5: Task set derived from Duty Cycling sensing task

they have to be supplied with a root node, that has its battery value set to the current battery charge (converted into battery slots). Additionally, it is of course possible to change the remaining predictions, but note, that the vector has to be the length as the amount of remaining time slots. Alternatively, in MCST N could also be left the same and the root node gets the current time slot as value, then the prediction vector should also have to have length N . The recalculation for DP works by solving the same ILP with new N and energy harvesting values, the solution quality and start of the assignment would then have to be found in the first column and using the current battery slot in the corresponding array. UD solves the problem differently, by providing a separate function that recalculates the assigned task in each slot.

For this, as well as the last experiment, the charge slots will from now on represent 1mAh each, so $B_{min} = 600, B_{max} = 1000$. $B_0 = 750$ was additionally set for the starting battery, because one test simulated more harvest than expected, which otherwise would cause energy overflows. The tasks also need to be adapted as a consequence. Therefore, as the last experiment should represent a real life application of the algorithms, they are now based on real values. Performance scaling for the tasks is achieved by Duty Cycling, specifically for a single sensing task. For this, a Grove Air Quality Sensor v1.3 was chosen, connected to the nRF52840 board and its active energy usage was measured at 105mA, while the passive energy drawn is 1mA (by sensor and board). One sensor activation takes 1s of time for initialization and taking the measurement. From these values, the tasks presented in Table 5.5 were derived, which use a fixed Duty Cycle. The quality was gained from the energy requirement by calculating $q_i = 2 \times \lfloor \sqrt{e_i} \rfloor$.

The test for the recalculation was conducted by first calculating an initial assignment and assuming it was executed until the 12th time slot. Then recalculations were performed, assuming that the harvested energy in the past and the future was (1) 20% / (2) 50% lower and (3) 20% higher than predicted in 3 separate simulations, so the battery level at the start of the recalculation was affected, as well as the energy predictions for the remaining time slots. Predictions were taken from PVGIS, historic

energy harvesting values for 15th of April 2023 (in order to have some more variation), then modified with $\pm 20\%$ and -50% and rounded. The simulation, differently from the usual approach, was started at midnight for this experiment. In Tables 5.6 (a)-(d) the results for the initial assignment, when assuming the predictions are accurate, as well as the recalculations if they are initiated after the 12th slot are shown. As both the initial and the recalculated schedule were computed by the specific algorithm that was tested, the battery charge values after the 12th time slot are different. Thus, this value (B_{12}) is displayed alongside the resulting quality and end battery value. It can be seen that DP in this case may not give an optimal result, Beam Search and UD are close up and MCTS is not too far off either. Every algorithm did at least find an energy neutral solution, proving the reliability of MCTS and Beam Search in this scenario.

Algorithm	B_{12}	End Quality	End Battery	B_{12}	End Quality	End Battery
DP	737	204	750	702	176	753
UD	711	204	756	676	176	765
MCTS	720	194	756	685	168	750
Beam Search	705	204	750	670	176	759

(a) Initial Assignment

(b) -20% Harvest

Algorithm	B_{12}	End Quality	End Battery	B_{12}	End Quality	End Battery
DP	651	130	752	771	216	758
UD	625	126	750	745	220	754
MCTS	634	122	750	754	216	753
Beam Search	619	122	752	739	220	758

(c) -50% Harvest

(d) +20% Harvest

Table 5.6: Comparison of algorithm results under different harvesting scenarios

5.5 Operation Simulation

The final test to which the algorithms are subjected is a simulation of the results for energy harvesting values of an entire year (January 1st until December 30th, so 355 days). The setup for this looks as follows; the algorithms are executed once every day, finding the assignment for the next 24 hours, at exactly 00:00. The energy predictions are taken from PVGIS for the entire year of 2023 and are assumed to be accurate, no recalculations are performed. Every day is divided into 24 time slots of 1 hour each,

to which the tasks shown in 5.5 are assigned. The charge values, again, are grouped into slots of 1mAh each, giving the battery parameters $B_{min} = 600$, $B_{max} = 1000$. B_0 , as before, is set to 750(mAh) as well. If an assignment calculated by an algorithm resulted in a higher battery level than B_0 , the charge could be carried over to the next day, meaning the next day's simulation starts with a higher battery than B_0 . However, the target energy level that should be achieved at the end of a day stayed constant, so this would just increase the possible budget for the next day. Again, the solar energy harvesting module described in the beginning of this chapter is considered as energy source, making this simulation similar to a real system, with the limitations and simplifications described earlier. The full simulations were performed for all four algorithms compared before, the full results as a heatmap can be found in the appendix. For each day, the accumulated quality and battery values of UD, MCTS and Beam Search were recorded and compared to those of DP. Therefore, a negative value for battery in the actual calendar depicting the battery difference does not mean that the solution is not energy neutral, but that the resulting battery charge was lower than that of DP for this day. Only energy neutral solutions were considered in general, as all algorithms found an energy neutral solution for every day. Additionally, the quality values calculated by all four algorithms for each day were summed up and also depicted in Table 5.7. According to the simulation results, MCTS may not be suitable for the specific simulation setup, at least when using solar energy. While it found a solution for every day of the year, they were mostly worse regarding quality, often even having more than 20% relative distance to the optimal. While it sometimes found better solutions than DP as a result of having saved more energy in the days before, it occurred much rarer than for the other two algorithms. UD and Beam Search were closer to DP, with Beam Search taking the edge in this particular simulation. Its solutions were rarely far away from the optimal, being 10-15% lower only 7 times. It also found a better or the same quality solution for more than half of the year, the quality accumulated over the entire year is also very close to DP.

Evaluation	MCTS	UD	Beam Search
<i>quality</i> higher than DP	20	89	179
<i>quality</i> 10-15% lower than DP	59	140	7
<i>quality</i> 15-20% lower than DP	101	5	0
<i>quality</i> >20% lower than DP	99	24	0
<i>quality</i> sum difference from DP	-14.98%	-7.24%	-1.11%
lower B_N than DP	43	41	72
10-25% higher B_N than DP	124	114	41
>25% higher B_N than DP	9	41	0

Table 5.7: Results comparing quality and end battery B_N to solutions from DP over the entire year of 2023

6 Conclusion

In this thesis, the problem of managing energy in energy harvesting sensor nodes of WSNs, while was investigated with the goal of maintaining constant operation and maximizing node performance. Multiple approaches were evaluated first, the decision then fell on the framework presented first by Kansal et al. [31], which revolves around finding an assignment of performance levels to time slots, over a concrete time span, that assures energy neutrality and maximizes overall performance. A graph model was developed for this purpose, which transferred the initial problem into a path finding problem in a DAG. A* and topological sorting were first introduced as possible solvers, finding optimal solutions to the problem. However, they offered worse performance than already existing perfect solvers, such as the DP [9] algorithm. Therefore, approximate heuristics were brought in, that could potentially provide a tradeoff between better performance and slightly worse result. Beam Search and MCTS were chosen for this, the latter having to be heavily modified to fit the graph structure and the particular problem. Since the problem description itself already relied on "perfect" energy harvest predictions, both algorithms also have the built in possibility of recalculating during a time span, if the actual incoming energy values diverge too much. As another possible comparison in addition to DP, the UD algorithm [17] was chosen, as it also provides an approximate solution. The algorithms were then compared and evaluated, mostly assuming solar energy harvesting, revealing in some limitations of MCTS regarding the start time of the time span. Moreover, the best parameters to run both algorithms were determined and used in all subsequent experiments. In the actual performance evaluations, it was visible that both approximate algorithms developed for this thesis have better execution time than DP, that also scales much better. The two optimal graph based algorithms on the other hand were slower than DP and, as this is the main advantage of using the graph based model, therefore excluded from the rest of the evaluation. Comparing to UD, MCTS is slightly worse in its execution time, while Beam Search was found to be about equal. A big downside of using the graph model is the memory necessary, but it may become less of a problem in the future, as there are already papers investigating the use of neural networks on wireless sensor nodes [67]. When scaling the problem parameters, both algorithms performed much better than the UD algorithm, always being within 15% of the optimal. It was additionally established that MCTS may be better suited to other means of energy harvesting (such as wind energy), where it yielded significantly better results than the other two heuristics. The result quality of all algorithms when recalculating was investigated as well, with all providing similar

solutions, MCTS was potentially a bit worse than the others in this regard. Finally, an experiment testing all algorithms with energy harvesting values from an entire year was conducted, which proved the robustness of all heuristics, as they all found a solution for each day. Furthermore it showed, that Beam Search may be superior to the other heuristics in regards to result quality. To conclude this thesis, it can be stated, that using an approximate, graph based algorithm might be sensible, when enough memory is available and execution time is to be limited. Beam Search in this case seems more suited to solar energy harvesting, while MCTS might be better for energy harvesting methods supplying a more constant stream of energy.

A Appendix

A.1 Beam Search and MCTS Graph Examples

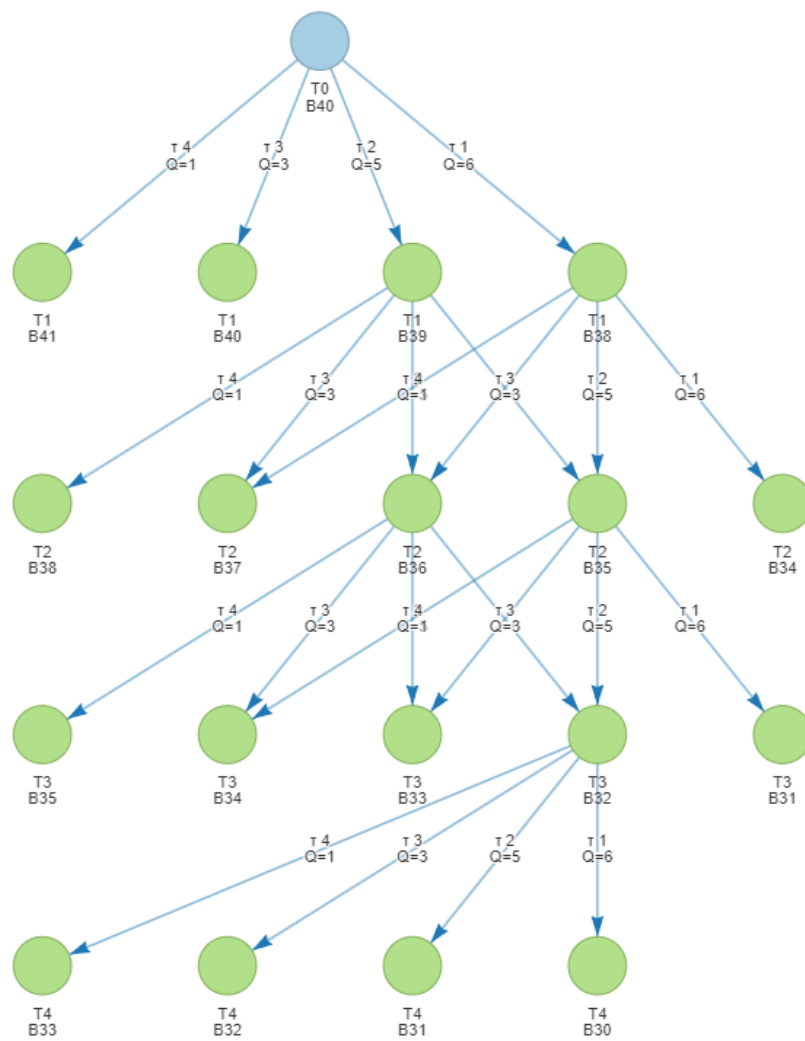


Figure A.1: MCTS Graph for $iterations = 6$

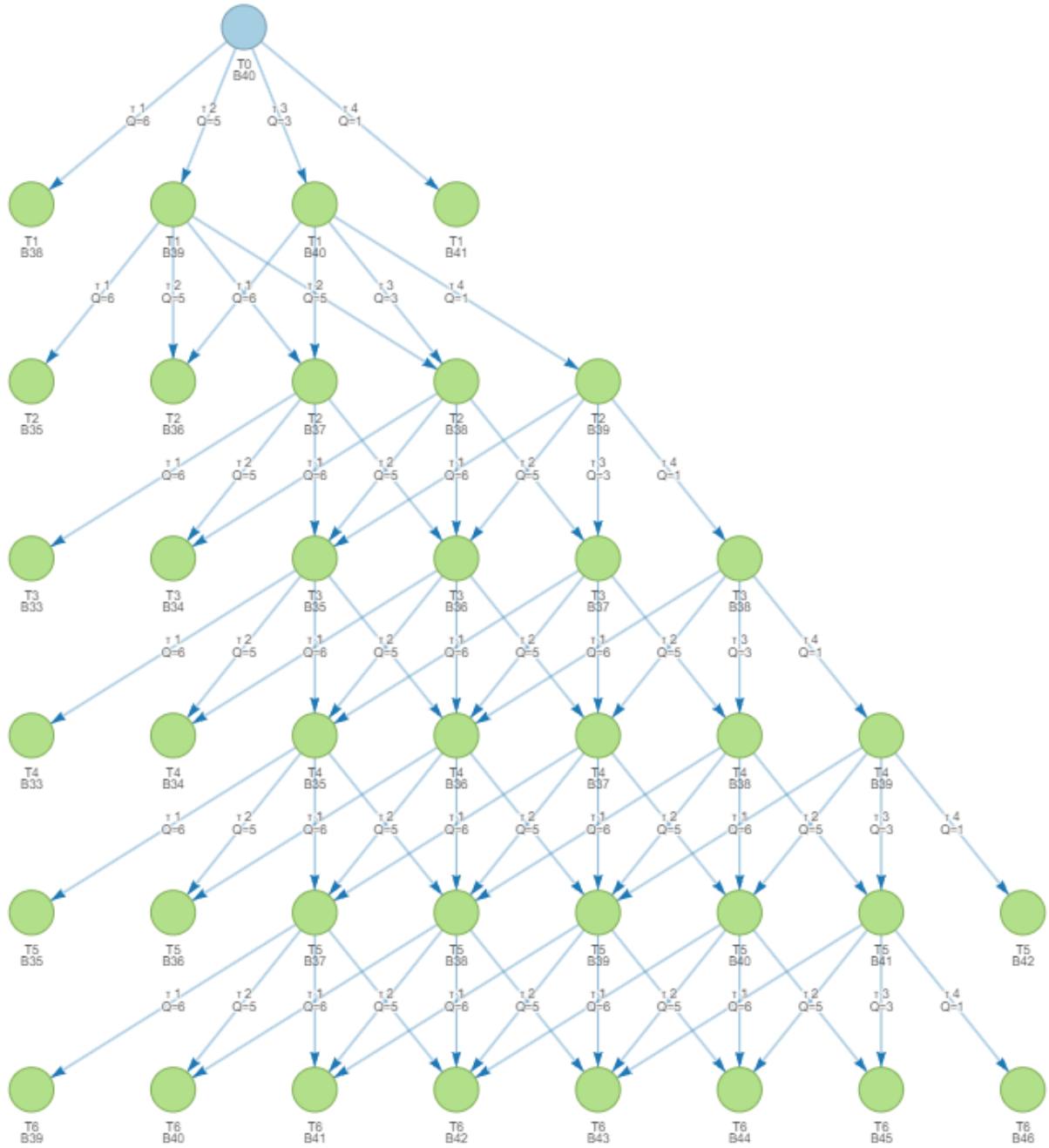


Figure A.2: Beam Search Graph for $beam_width = 2$.
Both graphs use $N = 6$, $E = [2, 0, 0, 2, 4, 6]$, $|\Theta| = 4$, $B_{min} = 30$, $B_{max} = 50$
and $B_0 = 40$

A.2 Operation Simulation Results

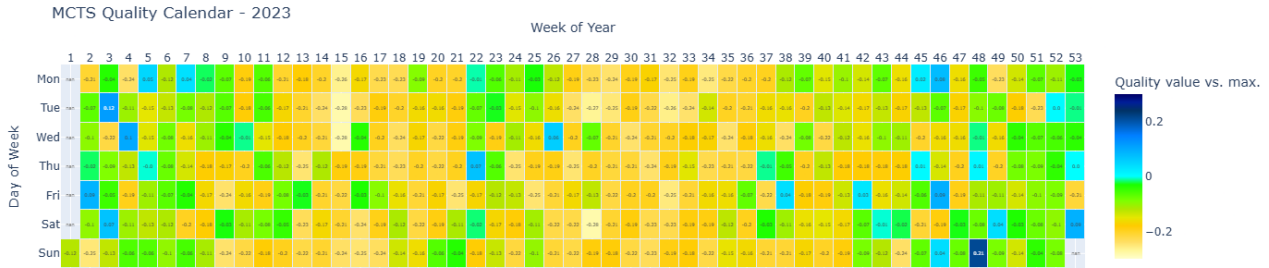


Figure A.4: Quality difference for every day of 2023 of MCTS compared to DP

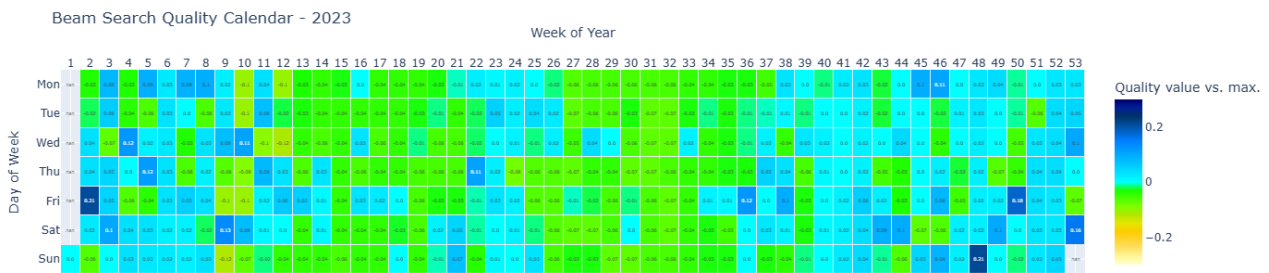


Figure A.5: Quality difference for every day of 2023 of Beam Search compared to DP

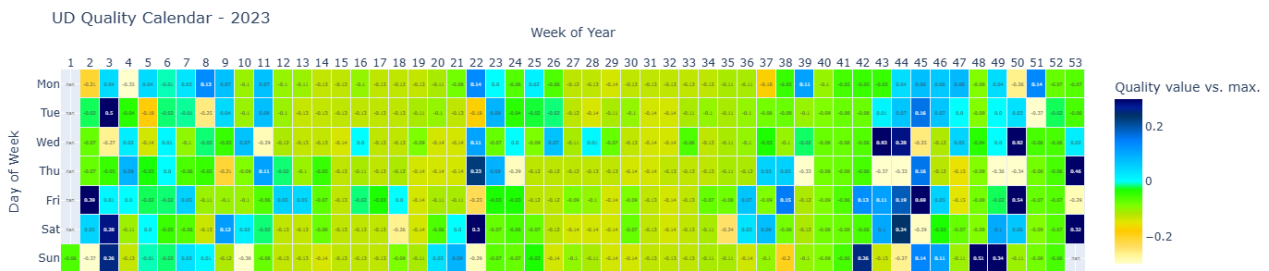


Figure A.6: Quality difference for every day of 2023 of UD compared to DP

A.2 Operation Simulation Results

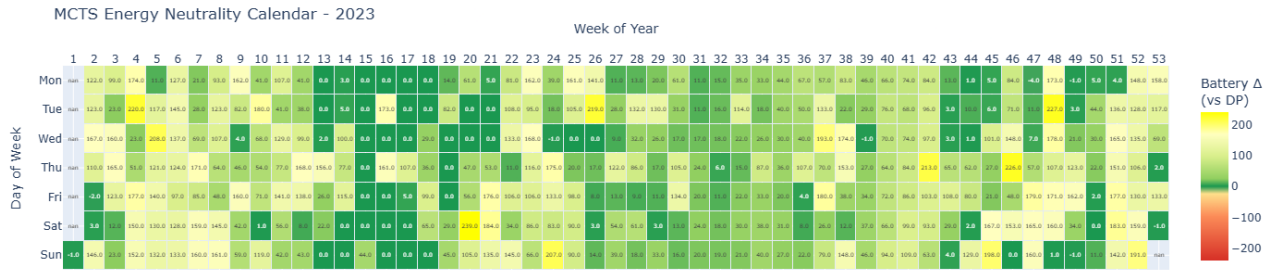


Figure A.7: Relative battery difference for every day of 2023 of MCTS compared to DP

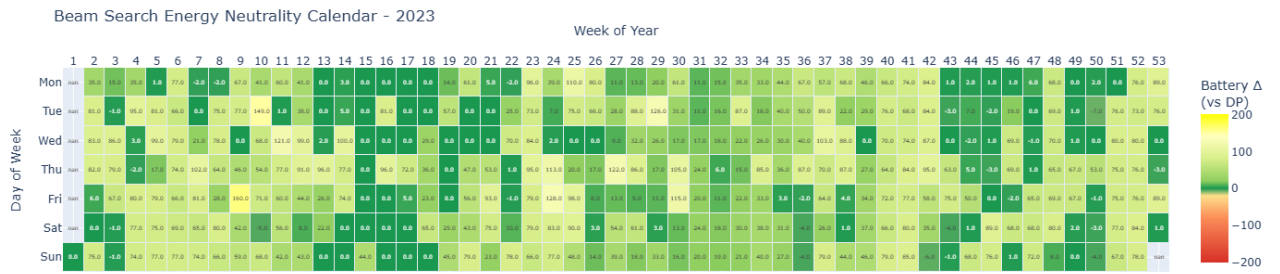


Figure A.8: Relative battery difference for every day of 2023 of Beam Search compared to DP

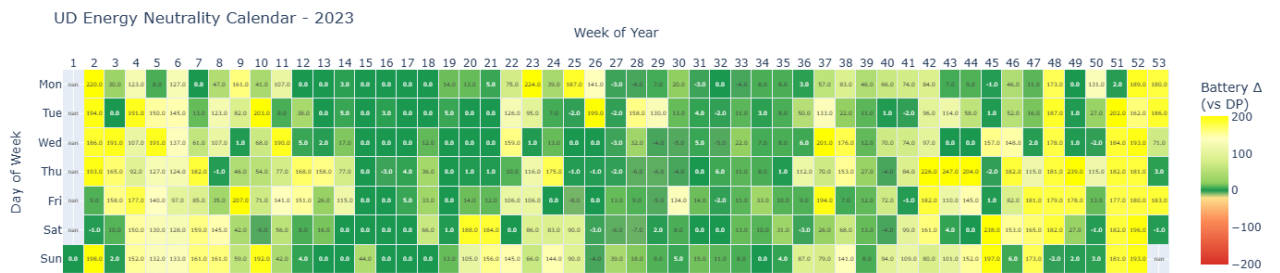


Figure A.9: Relative battery difference for every day of 2023 of UD compared to DP

A.3 Duty Cycle based Tasks

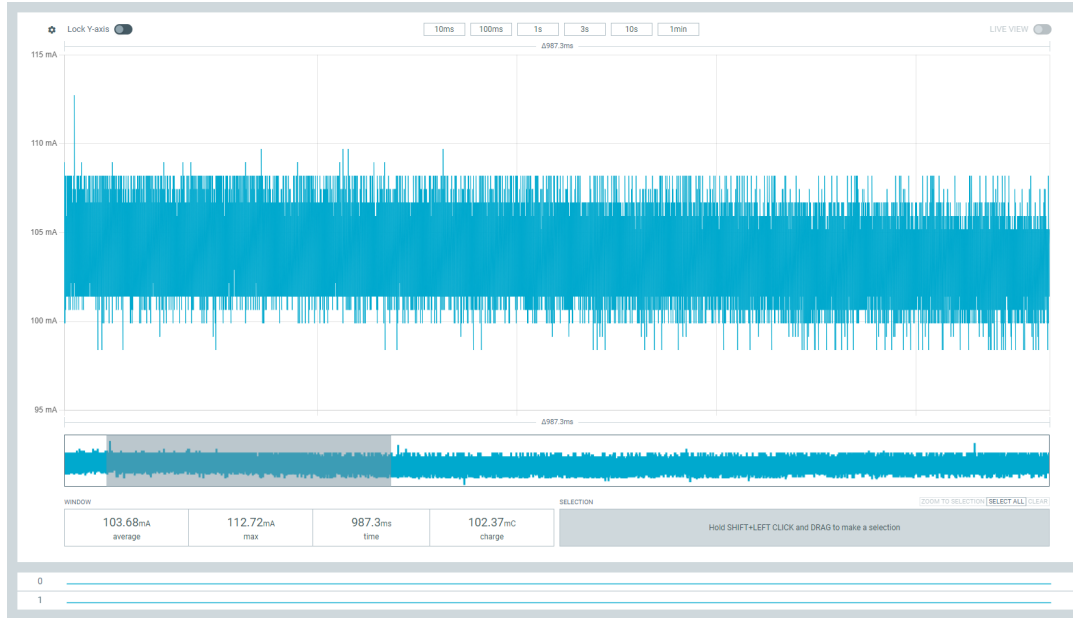


Figure A.10: Energy consumption of Grove Air Quality Sensor 1.3 while active.
Idle consumption is 1mAh

Example: Duty Cycle Calculation (25%)

For a duration of 1 hour, sensor is active for 1 second per measurement:

$$\text{Active phase: } 105 \text{ mA} \cdot 0.25 \text{ h} = 26.25 \text{ mAh}$$

$$\text{Idle phase: } 1 \text{ mA} \cdot 0.75 \text{ h} = 0.75 \text{ mAh}$$

$$\text{Total consumption: } 26.25 \text{ mAh} + 0.75 \text{ mAh} = 27 \text{ mAh}$$

$$\text{Quality: } 2 \times \lfloor \sqrt{27} \rfloor = 10$$

A.4 Penalty function parameter k

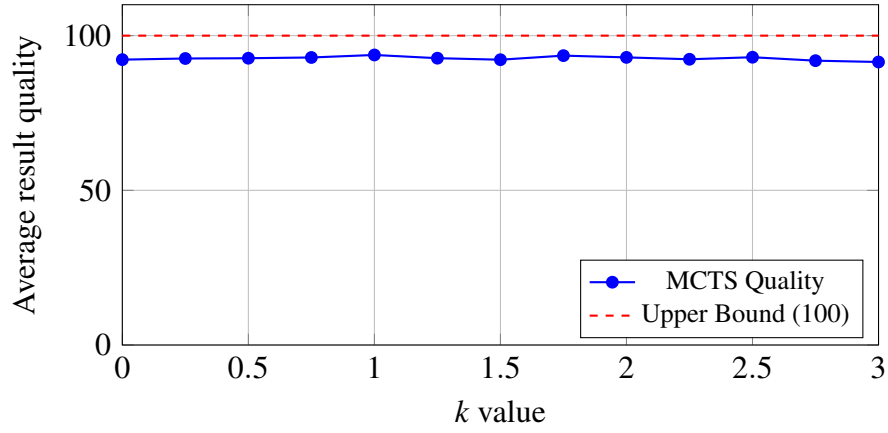


Figure A.11: Average solution quality (for varying k in MCTS penalty function)
 Peak at $k = 1$ with $quality = 93.75$
 12 Iterations, average of 100 executions

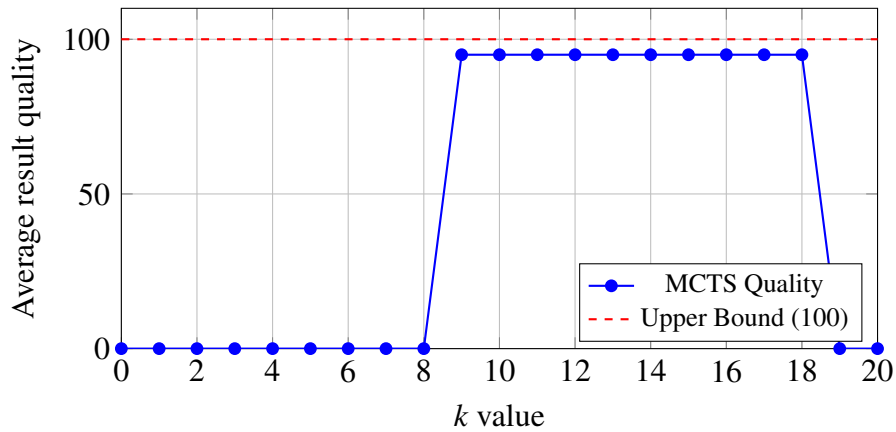


Figure A.12: Solution quality (for varying k in Beam Search penalty function)
 Peak at $k = 9$ to $k = 18$ with $quality = 95$
 $beam_width = 2$, simulation setups same as for Figure 5.1

Bibliography

- [1] Kemal Akkaya and Mohamed Younis. “Energy and QoS aware routing in wireless sensor networks”. In: *Cluster computing* 8.2 (2005), pp. 179–188.
- [2] Ian F Akyildiz, Tommaso Melodia, and Kaushik R Chowdury. “Wireless multimedia sensor networks: A survey”. In: *IEEE Wireless Communications* 14.6 (2007), pp. 32–39.
- [3] Ian F Akyildiz, Weilian Su, Yogesh Sankarasubramaniam, and Erdal Cayirci. “Wireless sensor networks: a survey”. In: *Computer networks* 38.4 (2002), pp. 393–422.
- [4] Kevin S. Anderson, Clifford W. Hansen, William F. Holmgren, Adam R. Jensen, Mark A. Mikofski, and Anton Driesse. “pvlib python: 2023 project update”. In: *Journal of Open Source Software* 8.92 (2023), p. 5994. DOI: 10.21105/joss.05994.
- [5] Dominik Bauer, Timothy Patten, and Markus Vincze. “Monte Carlo tree search on directed acyclic graphs for object pose verification”. In: *International Conference on Computer Vision Systems*. Springer. 2019, pp. 386–396.
- [6] Richard Bellman. “On a routing problem”. In: *Quarterly of applied mathematics* 16.1 (1958), pp. 87–90.
- [7] Carlo Bergonzini, Davide Brunelli, and Luca Benini. “Algorithms for harvested energy prediction in batteryless wireless sensor networks”. In: *2009 3rd International workshop on advances in sensors and interfaces*. IEEE. 2009, pp. 144–149.
- [8] Cameron B Browne, Edward Powley, Daniel Whitehouse, Simon M Lucas, Peter I Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. “A survey of monte carlo tree search methods”. In: *IEEE Transactions on Computational Intelligence and AI in games* 4.1 (2012), pp. 1–43.
- [9] Antonio Caruso, Stefano Chessa, Soledad Escolar, Xavier Del Toro, and Juan Carlos López. “A dynamic programming algorithm for high-level task scheduling in energy harvesting IoT”. In: *IEEE Internet of Things Journal* 5.3 (2018), pp. 2234–2248.

- [10] Antonio Caruso, Stefano Chessa, Soledad Escolar, Fernando Rincón, et al. “A Dynamic Programming schedule trading off quality and stability in task allocation for Energy-Neutral Internet of Things Devices Harvesting Solar Energy”. In: *ITU JOURNAL* 5.1 (2024), pp. 13–25.
- [11] Guillaume MJ-B Chaslot, Mark HM Winands, and H Jaap van Den Herik. “Parallel monte-carlo tree search”. In: *International Conference on Computers and Games*. Springer. 2008, pp. 60–71.
- [12] Rémi Coulom. “Efficient selectivity and backup operators in Monte-Carlo tree search”. In: *International conference on computers and games*. Springer. 2006, pp. 72–83.
- [13] David R Cox. “Prediction by exponentially weighted moving averages and related methods”. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 23.2 (1961), pp. 414–422.
- [14] Edsger W Dijkstra. “A note on two problems in connexion with graphs”. In: *Edsger Wybe Dijkstra: his life, work, and legacy*. 2022, pp. 287–290.
- [15] Soledad Escolar, Stefano Chessa, and Jesús Carretero. “Energy management in solar cells powered wireless sensor networks for quality of service optimization”. In: *Personal and ubiquitous computing* 18.2 (2014), pp. 449–464.
- [16] Soledad Escolar, Stefano Chessa, and Jesus Carretero. “Energy-neutral networked wireless sensors”. In: *Simulation Modelling Practice and Theory* 43 (2014), pp. 1–15.
- [17] Soledad Escolar, Stefano Chessa, and Jesus Carretero. “Optimization of quality of service in wireless sensor networks powered by solar cells”. In: *2012 IEEE 10th International Symposium on Parallel and Distributed Processing with Applications*. IEEE. 2012, pp. 269–276.
- [18] European Commission, Joint Research Centre. *Photovoltaic Geographical Information System (PVGIS)*. https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/pvgis-online-tool_en. Accessed: March 24, 2026. 2023.
- [19] Shimon Even. *Graph algorithms*. Cambridge University Press, 2011.
- [20] Emad Felemban, Chang-Gun Lee, and Eylem Ekici. “MMSPEED: multipath Multi-SPEED protocol for QoS guarantee of reliability and. Timeliness in wireless sensor networks”. In: *IEEE transactions on mobile computing* 5.6 (2006), pp. 738–754.
- [21] Andrea Gaglione, David Rodenas-Herraiz, Yu Jia, Sarfraz Nawaz, Emmanuelle Arroyo, Cecilia Mascolo, Kenichi Soga, and Ashwin A Seshia. “Energy neutral operation of vibration energy-harvesting sensor networks for bridge applications”. In: (2018).

-
- [22] Kai Geissdoerfer, Raja Jurdak, Brano Kusy, and Marco Zimmerling. “Getting more out of energy-harvesting systems: Energy management under time-varying utility with preact”. In: *Proceedings of the 18th International Conference on Information Processing in Sensor Networks*. 2019, pp. 109–120.
 - [23] Lars Hanschke and Christian Renner. “EmRep: Energy management relying on state-of-charge extrema prediction”. In: *IET computers & digital techniques* 16.4 (2022), pp. 91–105.
 - [24] Peter E Hart, Nils J Nilsson, and Bertram Raphael. “A formal basis for the heuristic determination of minimum cost paths”. In: *IEEE transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107.
 - [25] Tian He, John A Stankovic, Chenyang Lu, and Tarek Abdelzaher. “SPEED: A stateless protocol for real-time communication in sensor networks”. In: *23rd International Conference on Distributed Computing Systems, 2003. Proceedings*. IEEE. 2003, pp. 46–55.
 - [26] Jason Hsu, Sadaf Zahedi, Aman Kansal, Mani Srivastava, and Vijay Raghunathan. “Adaptive duty cycling for energy harvesting systems”. In: *Proceedings of the 2006 international symposium on Low power electronics and design*. 2006, pp. 180–185.
 - [27] Thomas Huld, Robert Müller, and Alessandro Gambardella. “A new solar radiation database for estimating PV performance in Europe and Africa”. In: *Solar Energy* 86 (2012), pp. 1803–1815. DOI: 10.1016/j.solener.2012.04.012.
 - [28] Ranjit Iyer and Leonard Kleinrock. “QoS control for sensor networks”. In: *IEEE International Conference on Communications, 2003. ICC’03*. Vol. 1. IEEE. 2003, pp. 517–521.
 - [29] Arthur B Kahn. “Topological sorting of large networks”. In: *Communications of the ACM* 5.11 (1962), pp. 558–562.
 - [30] Dionisis Kandris, Christos Nakas, Dimitrios Vomvas, and Grigorios Koulouras. “Applications of wireless sensor networks: an up-to-date survey”. In: *Applied system innovation* 3.1 (2020), p. 14.
 - [31] Aman Kansal, Jason Hsu, Mani Srivastava, and Vijay Raghunathan. “Harvesting aware power management for sensor networks”. In: *Proceedings of the 43rd annual design automation conference*. 2006, pp. 651–656.
 - [32] Aman Kansal, Jason Hsu, Sadaf Zahedi, and Mani B Srivastava. “Power management in energy harvesting sensor networks”. In: *ACM Transactions on Embedded Computing Systems (TECS)* 6.4 (2007), 32–es.

- [33] Aman Kansal, Dunny Potter, and Mani B Srivastava. “Performance aware tasking for environmentally powered sensor networks”. In: *Proceedings of the joint international conference on Measurement and modeling of computer systems*. 2004, pp. 223–234.
- [34] Aman Kansal and Mani B Srivastava. “An environmental energy harvesting framework for sensor networks”. In: *Proceedings of the 2003 international symposium on Low power electronics and design*. 2003, pp. 481–486.
- [35] Ulf Kulau, Johannes van Balen, Sebastian Schildt, Felix Büsching, and Lars Wolf. “Dynamic sample rate adaptation for long-term IoT sensing applications”. In: *2016 IEEE 3rd World Forum on Internet of Things (WF-IoT)*. IEEE. 2016, pp. 271–276.
- [36] Kris Lin, Jennifer Yu, Jason Hsu, Sadaf Zahedi, David Lee, Jonathan Friedman, Aman Kansal, Vijay Raghunathan, and Mani Srivastava. “Heliomote: Enabling long-lived sensor networks through solar energy harvesting”. In: *Proceedings of the 3rd international conference on Embedded networked sensor systems*. 2005, pp. 309–309.
- [37] Xiaoqiang Liu, Mingxue Li, Yufeng Zhang, Xiwen Zhu, and Yuesheng Guan. “Self-Powered wireless sensor node based on RF energy harvesting and management combined design”. In: *Energy Conversion and Management* 292 (2023), p. 117393.
- [38] R Lopes Pereira, J Trindade, F Gonçalves, L Suresh, D Barbosa, and T Vazão. “A wireless sensor network for monitoring volcano-seismic signals”. In: *Natural Hazards and Earth System Sciences* 14.12 (2014), pp. 3123–3142.
- [39] Bruce T Lowerre. *The harpy speech recognition system*. Carnegie Mellon University, 1976.
- [40] Xin Lu and Shuang-Hua Yang. “Thermal energy harvesting for WSNs”. In: *2010 IEEE International Conference on Systems, Man and Cybernetics*. IEEE. 2010, pp. 3045–3052.
- [41] Clara Meister, Tim Vieira, and Ryan Cotterell. “Best-first beam search”. In: *Transactions of the Association for Computational Linguistics* 8 (2020), pp. 795–809.
- [42] Nicholas Metropolis and Stanislaw Ulam. “The monte carlo method”. In: *Journal of the American statistical association* 44.247 (1949), pp. 335–341.
- [43] Edward F Moore. “The shortest path through a maze”. In: *Proc. of the International Symposium on the Theory of Switching*. Harvard University Press. 1959, pp. 285–292.

- [44] Tatsuya Morita, Masashi Fujiwara, Yutaka Arakawa, Hirohiko Suwa, and Keiichi Yasumoto. “Energy harvesting sensor node toward zero energy in-network sensor data processing”. In: *International Conference on Mobile Computing, Applications, and Services*. Springer. 2018, pp. 210–215.
- [45] Clemens Moser, Davide Brunelli, Lothar Thiele, and Luca Benini. “Lazy scheduling for energy harvesting sensor nodes”. In: *IFIP Working Conference on Distributed and Parallel Embedded Systems*. Springer. 2006, pp. 125–134.
- [46] Clemens Moser, Davide Brunelli, Lothar Thiele, and Luca Benini. “Real-time scheduling for energy harvesting sensor nodes”. In: *Real-Time Systems* 37.3 (2007), pp. 233–260.
- [47] Clemens Moser, Davide Brunelli, Lothar Thiele, and Luca Benini. “Real-time scheduling with regenerative energy”. In: *18th Euromicro Conference on Real-Time Systems (ECRTS’06)*. IEEE. 2006, 10–pp.
- [48] Clemens Moser, Jian-Jia Chen, and Lothar Thiele. “Dynamic power management in environmentally powered systems”. In: *2010 15th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE. 2010, pp. 81–88.
- [49] Clemens Moser, Jian-Jia Chen, and Lothar Thiele. “Power management in energy harvesting embedded systems with discrete service levels”. In: *Proceedings of the 2009 ACM/IEEE international symposium on low power electronics and design*. 2009, pp. 413–418.
- [50] Clemens Moser, Jian-Jia Chen, and Lothar Thiele. “Reward maximization for embedded systems with renewable energies”. In: *2008 14th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications*. IEEE. 2008, pp. 247–256.
- [51] Clemens Moser, Lothar Thiele, Davide Brunelli, and Luca Benini. “Adaptive power management in energy harvesting systems”. In: *2007 Design, Automation & Test in Europe Conference & Exhibition*. IEEE. 2007, pp. 1–6.
- [52] Clemens Moser, Lothar Thiele, Davide Brunelli, and Luca Benini. “Robust and low complexity rate control for solar powered sensors”. In: *Proceedings of the conference on Design, automation and test in Europe*. 2008, pp. 230–235.
- [53] Judea Pearl. “Heuristics: intelligent search strategies for computer problem solving”. In: (1983).
- [54] Joaquin Recas Piorno, Carlo Bergonzini, David Atienza, and Tajana Simunic Rosing. “Prediction and management in energy harvested wireless sensor nodes”. In: *2009 1st international conference on wireless communication, vehicular technology, information theory and aerospace & electronic systems technology*. IEEE. 2009, pp. 6–10.

- [55] Gregory J Pottie. “Wireless sensor networks”. In: *1998 Information Theory Workshop (Cat. No. 98EX131)*. IEEE. 1998, pp. 139–140.
- [56] Gregory J Pottie and William J Kaiser. “Wireless integrated network sensors”. In: *Communications of the ACM* 43.5 (2000), pp. 51–58.
- [57] Vijay Raghunathan, Aman Kansal, Jason Hsu, Jonathan Friedman, and Mani Srivastava. “Design considerations for solar energy harvesting wireless embedded systems”. In: *IPSN 2005. Fourth International Symposium on Information Processing in Sensor Networks, 2005*. IEEE. 2005, pp. 457–462.
- [58] Mohammad Rahimi, Hardik Shah, Gaurav S Sukhatme, John Heideman, and Deborah Estrin. “Studying the feasibility of energy harvesting in a mobile sensor network”. In: *2003 IEEE International Conference on Robotics and Automation (Cat. No. 03CH37422)*. Vol. 1. IEEE. 2003, pp. 19–24.
- [59] Karen Rose, Scott Eldridge, Lyman Chapin, et al. “The internet of things: An overview”. In: *The internet society (ISOC)* 80.15 (2015), pp. 1–53.
- [60] Cosmin Rusu, Rami Melhem, and Daniel Mossé. “Multiversion scheduling in rechargeable energy-aware real-time systems”. In: *15th Euromicro Conference on Real-Time Systems, 2003. Proceedings*. IEEE. 2003, pp. 95–104.
- [61] Abdallah Saffidine, Tristan Cazenave, and Jean Méhat. “UCD: Upper Confidence bound for rooted Directed acyclic graphs”. In: *Knowledge-Based Systems* 34 (2012), pp. 26–33.
- [62] Rahul C Shah and Jan M Rabaey. “Energy aware routing for low energy ad hoc sensor networks”. In: *2002 IEEE Wireless Communications and Networking Conference Record. WCNC 2002 (Cat. No. 02TH8609)*. Vol. 1. IEEE. 2002, pp. 350–355.
- [63] Faisal Karim Shaikh and Sherali Zeadally. “Energy harvesting in wireless sensor networks: A comprehensive review”. In: *Renewable and Sustainable Energy Reviews* 55 (2016), pp. 1041–1054.
- [64] Ashish K Shukla, Shirsendu Mitra, Shikha Dhakar, Arnab Maiti, Sudhanshu Sharma, and Krishna K Dey. “Electrochemical energy harvesting using microbial active matter”. In: *ACS Applied Bio Materials* 6.1 (2022), pp. 117–125.
- [65] Amit Sinha and Anantho Chandrakasan. “Dynamic power management in wireless sensor networks”. In: *IEEE Design & Test of Computers* 18.2 (2002), pp. 62–74.
- [66] Pierre-Louis Sixdenier, Stefan Wildermann, and Jürgen Teich. “GRES: Guaranteed Remaining Energy Scheduling of Energy-harvesting Sensors by Quality Adaptation”. In: *2024 13th Mediterranean Conference on Embedded Computing (MECO)*. IEEE. 2024, pp. 1–5.

- [67] Kay Smarsly, Kosmas Dragos, and Jens Wigenbrock. “Machine learning techniques for structural health monitoring”. In: *Proceedings of the 8th European workshop on structural health monitoring (EWSHM 2016), Bilbao, Spain*. 2016, pp. 5–8.
- [68] Sujesha Sudevalayam and Purushottam Kulkarni. “Energy harvesting sensor nodes: Survey and implications”. In: *IEEE communications surveys & tutorials* 13.3 (2010), pp. 443–461.
- [69] Robert Endre Tarjan. “Edge-disjoint spanning trees and depth-first search”. In: *Acta Informatica* 6.2 (1976), pp. 171–185.
- [70] Krishnaiyan Thulasiraman and Madisetti NS Swamy. *Graphs: theory and algorithms*. John Wiley & Sons, 2011.
- [71] Andrew Truitt and S Nima Mahmoodi. “A review on active wind energy harvesting designs”. In: *International Journal of Precision Engineering and Manufacturing* 14.9 (2013), pp. 1667–1675.
- [72] Chongfeng Wei and Xingjian Jing. “A comprehensive review on vibration energy harvesting: Modelling and realization”. In: *Renewable and Sustainable Energy Reviews* 74 (2017), pp. 1–18.
- [73] Liguang Xie, Yi Shi, Y Thomas Hou, and Andwenjing Lou. “Wireless power transfer and applications to sensor networks”. In: *IEEE Wireless Communications* 20.4 (2013), pp. 140–145.
- [74] Mohamed Younis, Moustafa Youssef, and Khaled Arisha. “Energy-aware routing in cluster-based sensor networks”. In: *Proceedings. 10th IEEE international symposium on modeling, analysis and simulation of computer and telecommunications systems*. IEEE. 2002, pp. 129–136.
- [75] Patrick Zippenfenig. *Open-Meteo.com Weather API*. 2023. DOI: 10 . 5281 / zenodo . 7970649. URL: <https://open-meteo.com/>.

Erklärung

Ich versichere, dass ich die Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Erlangen, March 24, 2026

Roman Riazanov